Formal Architecture and Machinery of the Normative Formal Core and Appendices G, I, J, K, and L

Stuart Swan, Platform Architect, Siemens EDA

18 August 2026

CORE/G Shared semantics + contract	I Local replay	J Safety glue	K Liveness	L Witness selection

Based on: Catapult HLS User View Scheduling Rules, dated 18 August 2026

The latest version of the scheduling rules document is available here:

https://github.com/Stuart-Swan/Matchlib-Examples-Kit-For-Accellera-Synthesis-WG/blob/master/matchlib_examples/doc/catapult_user_view_scheduling_rules.pdf

The scheduling rules document is the normative source: it defines the scheduling contract, formal models, assumptions, theorem statements, proofs, and evidence obligations. This formal architecture guide adds value by reorganizing that material into a coherent map of how the Core and Appendices G, I, J, K, and L depend on one another, what information crosses each proof boundary, and how design choices connect to the safety, witness-selection, and deadlock arguments. It therefore serves as a navigation and conceptual-integration aid rather than an alternate specification; when its simplified explanation and the scheduling rules differ, the scheduling rules remain authoritative.

Contents

1. Executive orientation

PURPOSE	**Provide a single mental model for the Normative Formal Core and five appendices that otherwise appear as separate layers of formal detail.** **Engineer takeaway:** Treat the architecture as a proof stack: the Core fixes the shared operational semantics; G states the observable contract; I generates local evidence; J constructs the global safety witness; K proves a separate conditional internal-deadlock result; and L selects source witnesses when latency-sensitive choices matter.

The formal architecture is best understood as two related, but deliberately separate, proof pipelines built on one qualified synchronous cycle model.

- **Safety pipeline (Core -> G -> I -> J).** Shows that a covered reachable RTL prefix has a matching reachable source prefix at the chosen observable boundary. Core.17 supplies the canonical ΣMatch occurrence/value relation and Appendix G expands it with E1-E5. On the normal compositional route, source-side Core.QSync/Core.QSync-Ref supplies the finite L2-settled snapshot required by J.RegPhaseNF/J.UnitExt; a directly validated J.GWR package may instead carry equivalent checked per-Horizon settling evidence. For the standard QSync route, J.CycleReach_post plus J.QSyncTraceInd first derive cycle-aligned J.TraceCertCov and J.1-Safe-QSyncCycle without importing the K-only Offer/KObs/drain obligations. Core.RTLCycleCarrier and J.CycleIdealCover then show that every reachable finite outer observable history is an ideal of one of those covered cycle-aligned histories, and J.1-Safe-QSync transfers the conclusion to all reachable finite RTL prefixes for properties satisfying J.IdealPrefixClosed. An already-proved J.QSyncCertInd projects to the safety induction. The generic J result remains finite-prefix reachability over Reach_post/Reach_ref and is observable trace compatibility rather than cycle-by-cycle or full-state equivalence.
- **Liveness/deadlock pipeline (Core -> G -> I -> J -> K).** Starts from the J-selected source/RTL package and its common typed Core.18 KObs record, reconstructs the wait-relevant observation at matched Core.KCut states, and then rules out the defined reachable closed internal message-passing wait-cycle cutpoint class under additional admissibility, fairness, progress, drain, environment, source-side, and certificate-coverage premises. The Policy-S reduction uses common K.OpKey keys, the K.CVPred/K.CV-WF provenance and comparison-availability closure, Core.FairPick/Core.FairCycleDovetail for the constructed fair continuation, K.EnvMF with the applicable K.JointFreeze-Ord/K.JointFreeze and K.CompareAdvanceLegal evidence, and the reset-first K.InterruptionGate scheme. In the standard QSync scope, Core.QSync-Post plus Core.QSync-KCutCanonical provide the complete canonical KCut domain; J.QSyncCertInd -> J.CertExist-QSync -> K.CertCov-QSync derives K.CertCov rather than requiring an unrelated pre-enumerated package set.
- **Witness-selection adapter (L).** Supplies the WC witness needed by I and J when failed non-blocking polls or other latency-sensitive epsilon choices affect later observable behavior. It selects an admissible source execution aligned with the free-running RTL; it does not redefine the source model or stall the RTL.

KEY ARCHITECTURAL FACT: Safety and liveness are not one theorem
Appendix J establishes a global observable-safety correspondence from local evidence. Appendix K does not strengthen that correspondence into liveness automatically. It adds a different argument with different assumptions: wait-for-graph reconstruction, drain discipline, fairness/progress, cutpoint coverage, and the scheduler-robust source-liveness premise A5-L.

1.1 What the formal framework is trying to achieve

The user-facing goal is a verification flow in which most functional verification and debug can remain at the pre-HLS SystemC level, while the RTL is checked against a precise implementation contract. The formal appendices make that promise conditional and auditable: they identify exactly which source rules, implementation properties, witness choices, boundary abstractions, and liveness assumptions must hold.

How to interpret the qualification detail. The formal architecture names many conditions that can look like new work when they are listed explicitly. The underlying engineering conditions were largely already present in conventional verification practice, both for HLS-generated RTL and for manually written or modified RTL. Reset behavior, boundary and capacity correctness, request persistence, cycle-local settling, arbitration and progress behavior, environment assumptions, and the mapping from intended interface semantics to implementation all have to be valid if source- or specification-level conclusions are to be trusted. The framework makes those dependencies explicit, separates their ownership, and requires evidence for the subset consumed by the selected claim. It can therefore add assurance and explicit accounting without implying that every named condition is a new design-user task.

The central idea is to compare only architecturally meaningful observations: committed message transfers, synchronization operations, and anchored signal reads/writes at selected boundaries. Internal behavior outside Σ is not all represented by the same formal object. Source evaluation and other proof-hidden micro-steps may be ε-steps; same-cycle settling is classified separately as δ-settling; a real clock cycle with hidden registered progress but no selected Σ event is a silent τ-cycle; and a B6 idle tick performs no ε work. Section 4 distinguishes these cases.

1.2 What the framework does not claim

- It does not claim identical cycle timing between pre-HLS and post-HLS models.
- It does not claim equality of all internal state, requests, or pipeline registers.
- It does not automatically prove that a particular HLS run or RTL instance conforms to E3, E4, E5/Core.SyncFence, the Core.IC request-lifecycle/attribution invariant, B-faithfulness, Core.RegSeq, Core.ReachPrep, Core.ElabProj, Core.MemSyncOrder, the Core.QSync structural conditions including QS3 realization/adequacy and, for the standard RTL K-facing domain, QS2a origin recoverability, or the other implementation obligations. Core.IC and Core.SyncFence are constitutive legality/invariant rules of Sys_ref; their corresponding RTL behavior is separately evidence-backed rather than obtained by definition. In the standard QSync scope, Core.KCutClosure_post and canonical KCut identification are theorem-derived from accepted structural settling evidence rather than supplied as arbitrary normalization assumptions.
- It does not infer global deadlock freedom from the source-order no-reverse rule alone.
- The standard compositional QSync route, and the standard K-facing/universal G-K results, do not cover unqualified non-QSync settling semantics. Pure combinational feedback that is not broken by explicit registered or finite-capacity state, non-deterministic or order-dependent same-cycle settling, and multi-clock instances without a separately specified semantic layer are outside that standard scope. Plain finite-prefix J.1 may instead use a directly validated J.GWR package carrying the required checked per-Horizon L2-settling evidence. A non-QSync design is not thereby asserted incorrect; broader K-facing or universal use requires an explicitly defined and separately qualified theorem extension.
- **It does not treat** an arbitrary finite or unqualified Policy-S simulation as a complete liveness proof. The primary route requires one selected complete instrumented run plus separately established SDet, K.OpKey/K.CVPred/K.CV-WF provenance and comparison-availability, route applicability, K.EnvMF and any needed JointFreeze/CompareAdvanceLegal evidence, Core.CycleClosure with the constructive Core.FairPick/Core.FairCycleDovetail discharge of Core.IssueFair/B2/B3, the reset-first interruption gates (or the uniform U1-U3 discharge), total finite-bound response coverage, response cleanliness, and certificate validity. For the standard universal RTL conclusion, audited QSync plus J.QSyncCertInd derive both the canonical KCut domain and K.CertCov; a single observed RTL or Policy-S run does not establish that universal package totality.
- It does not make latency-sensitive polling safe merely by hiding failed polls as epsilon steps; such sites require NB-CF, isolation, direct witness construction, or Appendix L snooping.

2. Architecture at a glance

PURPOSE	**Show the dependency chain from source/RTL semantics to the final safety and liveness conclusions.** **Engineer takeaway:** The load-bearing seam is between J and K: J exports one equal KObs record for matched cutpoints; K reasons only through that compact observation plus separate admissibility and liveness evidence.

NORMATIVE FORMAL CORE + APPENDIX G - SHARED SYNCHRONOUS SEMANTICS AND CONTRACT

Core: Sys_ref, Core.8 abstract E4 Commit/Commit(q), one-global-clock scope, Core.RegSeq/Core.ReachPrep/Core.IC, Core.PolicyL/PolicyS and Core.IssueFair, Core.Phase/Core.Settle/Core.QSync (including RTL QS2a), Core.CycleState/Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure, Core.FairPick/Core.FairCycleDovetail for constructed fair continuations, Core.SyncFence, Core.MemSyncOrder, BoundaryEvalPoint/Core.KCut/Core.QSync-KCutCanonical, Core.ElabProj/Core.WFG/Core.16/Core.Drain/Core.SettleKBridge, Core.17 ΣMatch, Core.18 typed KObs correspondence, and the downstream KCut-closure interface. Core.IC/Core.SyncFence are source legality rules with separate RTL conformance obligations. G: observable alphabet, source rules R1-R5, and compatibility obligations E1-E5.

APPENDIX I - LOCAL PROOF

Logical replay checkpoint, operational preparation/issue evidence, dynamic-operation identity, deferred NB holes

APPENDIX L - WITNESS ADAPTER

When needed, select epsilon outcomes from free-running RTL to discharge WC and align latency-sensitive choices

APPENDIX J - GLOBAL SAFETY CONSTRUCTION

LocalGWR -> generated D1-D4 semantic dependencies + D5 Horizon stratification -> J.HorizonOrderSafe/DepRank -> validated finite-ideal order -> registered phase-normal replay -> J.GWR-Comp -> J.0 -> Theorem J.1. For standard QSync safety, J.QSyncTraceInd -> J.TraceCertCov-QSyncCycle -> J.1-Safe-QSyncCycle establishes cycle-aligned packages; Core.RTLCycleCarrier -> J.CycleIdealCover -> J.1-Safe-QSync extends J.IdealPrefixClosed properties to all reachable finite outer observable prefixes. The richer J.QSyncCertInd used by K projects to the same safety induction.

SAFETY OUTPUT

Post -> Ref matching and J.1-Safe transfer of J.IdealPrefixClosed observable safety properties. On CycleReach_post, QSyncTraceInd supplies cycle-aligned packages without K-only Offer/KObs/drain evidence; CycleIdealCover then gives the all-finite-prefix QSync safety conclusion without per-prefix packages.

KOBS BRIDGE

Common typed Core.18 record <E,w,H,O>; J constructs KObsPost_J and J.KObsConf proves equality for the exact selected source/RTL package

APPENDIX K - CONDITIONAL LIVENESS

Equal Core.18 KObs at matched KCuts -> J reconstruction -> K deadlock factoring -> one certified KCut -> A5-L contradiction. The primary Policy-S route adds K.Low.A5, common K.OpKey keys, K.CVPred/K.CV-WF plus comparison-availability, K.EnvMF with JointFreeze where needed, K.CompareAdvanceLegal, Core.CycleClosure/Core.FairPick/Core.FairCycleDovetail, and reset-first K.ResetEpoch -> K.TerminalDisposition interruption handling. Universal RTL preservation consumes Core.KCutClosure_post plus K.CertCov; in the standard QSync route Core.QSync-Post/Core.QSync-KCutCanonical and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync derive those domain/package facts. Appendix K-P supplies the paper reduction; Lean mechanization, reusable certificate checking, and concrete tool/flow qualification remain incomplete.

APPENDIX M.7b PACKAGING OVERLAY - standard QSync certificate profile
Four logical interfaces package the M.7a ledger without replacing it: (1) instance and QSync qualification; (2) J correspondence/refinement; (3) K applicability/provenance; and (4) Policy-S response. The names are packaging interfaces, not new theorem predicates or required physical files. Every artifact remains bound through K.CertHdr/K.CertPkg. Standard J.IdealPrefixClosed QSync safety uses bundles 1-2; the primary universal Policy-S K route uses all four; an alternate A5-L discharge replaces bundle 4 with the corresponding K.A5Cert route payload.

2.1 The main interfaces between layers

Interface	What crosses it
Core -> G/I	The Core supplies Sys_ref, Core.8 abstract E4 Commit/Commit(q), dynamic operations and explicit abstract boundaries, Core.ReachPrep and the Core.IC request lifecycle, the single-clock/QSync same-cycle model including RTL QS2a where K-facing, Core.RegSeq/Core.SyncFence/Core.MemSyncOrder, Policy L/S and Core.IssueFair, Core.Phase/Core.LAdm, Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure, Core.FairPick/Core.FairCycleDovetail for constructed complete continuations, Core.Settle/Core.KCut/Core.QSync-KCutCanonical, Core.ElabProj/Core.WFG/Core.16/Core.Drain/Core.SettleKBridge, Core.17 ΣMatch, the Core.18 KObs type/equality interface, and downstream KCut-closure semantics. Core.IC/Core.SyncFence are constitutive on Sys_ref; G/E3/E5 plus mapping/attribution evidence establish the corresponding RTL conformance. G constrains the shared model through R1-R5/E1-E5; I consumes the Core semantics and G/E3/E5 discipline.
I -> J	I supplies process-local J.PCert evidence: a logical replay checkpoint plus separate operational preparation/issue facts, exact dynamic identity, typed footprints, and deferred non-blocking holes. These local facts intentionally do not claim global channel enablement or one common source execution.
L -> I/J	L supplies a witness selector proving WC when epsilon-modeled choices can affect later observable control, values, request attribution, or frontier identity.
J -> K	For one cutpoint, J supplies a reachable source witness, theorem-derived phase reachability, exact NormRef/NormPost KCut endpoints, and a common typed record KObsPost_J(rho_post;E,w,H,O) = KObs_E,w(sigma_ref), established by J.KObsConf after HCanon/OfferReach/OfferConf; K.DrainReach separately carries Core.SettleKBridge. For universal standard-QSync coverage, J.QSyncCertInd carries that substantive package across every reachable cycle and J.CertExist-QSync/K.CertCov-QSync derive K.CertCov. K factors its own deadlock, waiver, terminal, and diagnostic predicates and never compares arbitrary RTL/source microstates.

Interface	What crosses it
Policy-S -> K	The design-user activity is one selected complete instrumented Policy-S execution. The flow separately establishes SDet; K.Low/K.Low.A5; common K.OpKey and unconditional serial-route K.CVPred/K.CV-WF provenance including comparison-availability; K.EnvMF with K.JointFreeze only for admitted StandardEnv multi-frontiers; K.CompareAdvanceLegal (ordinary or comparability-certified singleton cases use K.CompareAdvanceLegal-Ord); Core.CycleClosure plus the constructive Core.FairPick/Core.FairCycleDovetail discharge; K.DomResetScope and either per-candidate nested K.ResetEpoch/K.TerminalDisposition gates or one uniform U1-U3 gate record; K.CompleteS; total finite-bound K.RespCov; K.RespClean; and certificate validity. For a universal RTL conclusion the preferred standard route also supplies checked QSync/QSyncCertInd evidence from which K.CertCov is derived. The paper reduction is supplied; Lean mechanization and flow qualification remain outstanding.

2.2 Generic theorem machinery versus design-specific evidence

The framework deliberately separates proof algorithms that should be mechanized once from evidence that must be supplied or checked for each design, theorem instance, or RTL build.

Category	Examples
Reusable formal machinery	Definitions of Core.17 trace compatibility/Core.18 typed KObs correspondence, local-to-global construction, D1-D4/D5 dependency stratification, finite-ideal induction, phase normalization, Core.QSync-Ref/Core.QSync-Post/Core.QSync-KCutCanonical, Core.RTLCycleCarrier, J.IdealPrefixClosed/J.CycleIdealCover/J.1-Safe-QSync, J.QSyncCertInd-Trace, J.TraceCertCov-QSyncCycle, J.CertExist-QSync and K.CertCov-QSync, Core.CycleResult/Core.CycleChoiceExt, Core.FairPick/Core.FairCycleDovetail, Core reconstruction functions, J.KObs reconstruction, and K.KObs factoring. Appendix K-P supplies K.Low.A5, K.OpKey/K.CVPred/K.CV-WF, K.JointFreeze-Ord/K.CompareAdvanceLegal-Ord, K.2b.P0/P1 and the remaining Policy-S serial-dominance/reduction paper proofs. Lean mechanization, reusable certificate validation, tool/flow qualification, and concrete proof-producing QSync/FairPick/certificate generators remain reusable implementation work.
Per-design source evidence	Source well-formedness, unique signal anchors, Core.RegSeq classification, ReachPrep coverage where eager preparation is admitted, source semantics using the constitutive Core.IC/Core.SyncFence legality rules, source-side QSync structural conformance or direct per-Horizon settling evidence, WC/NB-CF/snooping coverage, Core.MemSyncOrder schedule/mapping qualification where array accesses cross synchronization, J.Mem shared-memory lowering when cross-process value flow exists, barrier well-formedness, and value/control provenance. Qualified Sys_B^elab templates may discharge reusable settling facts, but the whole-source composition/adequacy check remains instance-owned.

Category	Examples
Per-RTL/tool evidence	E3/E5 scheduling conformance including the RTL counterparts of Core.IC request lifecycle/attribution and Core.SyncFence; Core.8 boundary-commit mapping from abstract E4 events to mapped valid/ready/payload observations; E4 channel realization; exact capacities/boundaries; B-faithfulness; Core.ElabProj; complete request attribution; reset behavior; ClockScope; and RTL-side QSync frame/finite-acyclic deterministic-settling/boundary-completeness plus QS3 realization/adequacy. In the standard K-facing scope this also includes QS2a K-origin recoverability for every reachable RTL KCut. A universal QSync route must mechanize or independently check the base/one-cycle/exhaustive-successor J.QSyncCertInd invariant; the safety-only projection may instead use J.QSyncTraceInd.
Per-liveness-instance evidence	The fixed capacities/elaboration/stimulus/environment/witness, simulator semantics, Policy-S run identity, common OpKey/CVFact universes and well-founded provenance/comparison-availability certificate, K.EnvMF plus any required multi-frontier K.JointFreeze and CompareAdvanceLegal evidence, Core.CycleClosure/Core.FairPick/B5 qualification, K.DomResetScope/DomEpoch with either candidate-specific nested gates or a uniform U1-U3 record, response-monitor coverage/bounds, run completeness, exact KCut correspondence packages, QSync/QSyncCertInd provenance for theorem-derived Core.KCutClosure_post and K.CertCov, and the selected A5-L discharge route.

3. Design choices that determine proof applicability

PURPOSE	**Connect common HLS design patterns to the proof layer and evidence route they affect.** **Engineer takeaway:** The architect does not construct the formal certificates, but the design must expose communication, timing, reset, and state-transfer behavior clearly enough for the tool and verification flow to construct them. Appendix F of the scheduling-rules document is the architect-facing checklist; this section explains why those choices matter to the proof architecture.

This guide remains explanatory. Appendix F and the normative sections it references remain authoritative. The purpose here is to connect recognizable design patterns to Core, G, I, J, K, and L without requiring an architect to begin from certificate or theorem names.

3.1 The architect/flow responsibility boundary

ARCHITECT / FLOW BOUNDARY

The architect chooses interfaces, reset topology, memory communication, buffering, and protocol structure that admit the required correspondence. The tool and verification flow construct and check the detailed replay, channel, attribution, footprint, reset, coverage, and liveness evidence. A design may synthesize and function correctly while remaining outside a selected theorem instance when proof-relevant behavior is implicit or modeled at the wrong boundary.

The useful question is therefore not whether the architect can write a certificate. It is whether the chosen architecture exposes enough explicit structure for a qualified flow to determine what happened, assign each request or value transfer to the correct dynamic operation and reset epoch, and compare the same abstract channel and observation boundaries in source and RTL. The G-K theorem stack is a single-global-clock formalism: a selected theorem instance must stay within one synchronous clock domain unless a separate multi-clock semantic layer is supplied; CDC structures are therefore outside the ordinary instance or must be represented by separately verified boundaries. Core.IC and Core.SyncFence illustrate this boundary directly: they define legal behavior of the prescribed source transition system, while the RTL/tool flow must prove the mapped request lifecycle, process-facing completion, and synchronization-fence behavior through the corresponding conformance evidence.

3.2 Design choices affecting the safety and correspondence pipeline

These choices affect the Core -> G -> I -> J pipeline and the Post -> Ref safety result. They are not merely coding-style preferences: each determines whether the source and RTL provide a common observable contract and whether the local replay records can be composed into one reachable source witness.

Qualified synchronous settling discipline. (Core.RegSeq, Core.Settle, Core.QSync, Appendix Q)

- **Design implication.** Treat the selected theorem instance as ordinary synchronous hardware: one global clock; registered process/pipeline/channel state; sequential request/payload outputs determined from registered state and operands fixed for the cycle; and a complete same-cycle combinational/handshake network that is finite, deterministic, and acyclic after cutting at explicit state boundaries. The accepted transition relation must actually be realized by that qualified settling network. For RTL in the standard K-facing scope, every reachable post KCut must additionally carry QS2a origin recoverability: a unique same-cycle registered/input frame and designated reachable K-origin connected to that KCut only by qualified settling before the next registered update.
- **Why the proof cares.** The L2 frame freezes registered state and selected cycle-t exogenous inputs while delta-settling computes the common boundary snapshot. Core.QSync-Ref makes source SettlePoint existence/uniqueness a theorem. Core.QSync-Post supplies the RTL KCut/closure interface, and QS2a lets Core.QSync-KCutCanonical prove that every reachable standard-scope RTL KCut is one of those canonical same-cycle endpoints. This canonical domain is what the J.QSyncCertInd/K.CertCov-QSync package-totality route ranges over.
- **Supported alternatives.** Pure combinational feedback and unqualified non-QSync settling semantics are outside the standard compositional QSync and K-facing/universal G-K scope. Plain finite-prefix J.1 may still use a directly validated J.GWR package carrying the required checked per-Horizon L2-settling evidence. A separately defined and qualified

extension may target the downstream Core.KCutClosure interface for broader non-QSync use. Tool reports are evidence only to the extent they are trusted or independently checked; accepting an unverified settling claim enlarges the declared trusted base.

Observation boundaries and signal timing. ***(Appendix F: safety items 1-3)***

- **Design implication.** Give every sequential signal read and write one unambiguous synchronization anchor. Keep direct inputs stable for the applicable reset epoch or update them through the supported synchronization handshake. Environment and testbench decisions used by the proof should depend on prior observable causal history, not on an unmodeled absolute cycle count.
- **Why the proof cares.** Appendix G defines the selected signal observations, Appendix I replays the process state at those anchors, and Appendix J composes complete observable units. An ambiguous branch-dependent anchor or a hidden latency-triggered environment decision does not identify one stable observation for the construction to match.
- **Supported alternatives.** Move the signal operation to a unique boundary, use hls_direct_input_sync, expose the timing decision through an explicit protocol, isolate a cycle-accurate or latency-sensitive block, or verify the timing-dependent behavior directly at RTL rather than assuming it transfers from an arbitrary source run.

Non-blocking calls and invisible polling behavior. ***(Appendix F: safety items 4-5)***

- **Design implication.** A failed PopNB or PushNB may be hidden from the committed observable trace, but it is not harmless when its result or cadence determines arbitration, retries, timeout counters, payloads, endpoint choice, or the next visible action. An unbounded loop of failed polls is also not a stable wait in the ordinary replay fragment.
- **Why the proof cares.** Appendix I may carry a deferred non-blocking decision only past work proved independent through NoDependentUse. Appendix J later makes the shared snapshot decision. When the outcome selects later observable behavior, WC must identify a realizable source witness; Appendix L is the common implementation-alignment route.
- **Supported alternatives.** Prefer blocking communication when the architecture permits it. Otherwise prove NB-CF, use a valid Appendix L WC/L.Dir witness, directly construct the witness, or isolate and explicitly model the polling logic as a timing-sensitive protocol block.

Shared state and the actual communication boundary. ***(Appendix F: safety items 6-8)***

- **Design implication.** Cross-process shared-memory value flow must satisfy the J.Mem lowering condition: every dependency through which one process's write can affect another process's observable behavior is made visible through covered channel or anchored-signal operations. Separately, whenever an array/memory access in a process is source-ordered with a selected synchronization call, Core.MemSyncOrder requires the HLS schedule/mapping layer to preserve the stated commit-time fence (source-before accesses commit no later than the sync; source-after accesses commit strictly later). Core.MemSyncOrder is a scheduling-contract qualification check, not an Appendix I/J replay event or a replacement for J.Mem.
- **Why the proof cares.** The present J invariant does not carry an independent global shared-memory state, and E4 channel reasoning is meaningful only at a boundary whose enablement, occupancy, and transfer semantics match RTL. Treating a fall-through FIFO as an ordinary non-fall-through FIFO, or treating joined input capacities as independent slack, can produce the wrong replay and wait-state reconstruction.
- **Supported alternatives.** Use the supported memory/channel libraries and mapping layer, provide the separate memory-state extension, model a bypass as an explicit rendezvous path, or select Sys_B^elab with staged, bundle, join, coupler, guard, or epoch-gate boundaries and a checked Core.ElabProj package.

Reset topology and cross-epoch transaction disposition. ***(Appendix F: safety item 9)***

- **Design implication.** Reset both endpoints and the state of a live channel consistently, or define an explicit flush, abort, cancellation, or end-of-epoch protocol. Outstanding requests and in-flight data cannot be silently retained, discarded, or reattributed across a reset boundary.
- **Why the proof cares.** Appendix J represents reset as an atomic observable unit with a named scope, fresh dynamic identities, request clearing, reset-state correspondence, and framing outside the scope. Appendix K adds a second question: whether a reset can occur during the serial comparison without invalidating the pending operation used by the reduction.
- **Supported alternatives.** Use a channel-consistent reset scope, make cross-epoch disposition an explicit observable protocol, or supply a separately verified reset-state extension. For the Policy-S deadlock route, either provide candidate-

specific K.DomResetScope plus reset-first K.ResetEpoch/K.TerminalDisposition gate records, or use the normative uniform instance-level U1-U3 discharge: one conservative dominance scope covering all potentially relevant processes/boundaries, no permitted reset intersecting it, and no relevant pending frontier at any reachable DeclaredTerminal state. If neither route can be certified, use another A5-L route.

3.3 Additional design choices affecting the deadlock pipeline

The following choices affect the additional Core -> G -> I -> J -> K liveness pipeline. They are not extra premises for every use of the Appendix J safety theorem. They matter when the claim is that the defined reachable closed internal message-passing wait-cycle cutpoint cannot occur.

Unambiguous wait ownership and persistent requests. ***(Appendix F: deadlock items 1-3)***

- **Design implication.** Each analyzed logical channel should have one producer and one consumer, with shared buses and multi-client resources represented by explicit arbiters and point-to-point channels. Appendix K's deadlock observation boundary must include every WFG-relevant explicit abstract message-passing boundary; RTL micro-staging already absorbed into the selected B(c) remains internal. Multi-lane or burst behavior must be explicit. Once an ordinary blocking request is issued, Core.IC requires the same attributed dynamic instance (and stable Push payload) to remain pending until process-facing commit or an affecting RW2 reset; inserted latency is permitted under continued-request attribution. Independent nonterminal cancellation, withdrawal, retry, or reattribution is outside the present G-K theorem model.
- **Why the proof cares.** Core.WFG assigns each blocked frontier item to one complementary endpoint owner. K also reconstructs a typed residual offer for one dynamic operation. Hidden arbitration or a speculative request that can disappear does not provide the stable edge and attribution used by the wait-cycle theorem.
- **Supported alternatives.** Insert an explicit arbiter process, expose lanes or beats in the abstract interface, or provide a custom shared-resource ownership and arbitration proof instead of treating the resource as an ordinary scalar point-to-point channel.

Drain-only behavior after a blocking frontier is reached. ***(Appendix F: deadlock item 4)***

- **Design implication.** Previously accepted pipeline work may finish and release finite downstream state, but later blocking work must not continue entering behind a disabled frontier or replenish the drain indefinitely. A synchronization call may advance into the next source interval only after every earlier blocking operation in pref_S(s) has process-facing committed; same-cycle message/sync completion is allowed under Core.SyncFence.
- **Why the proof cares.** Core.16 activates at the L2 SettlePoint. Lemma Core.SettleIsKCut makes that settled source state a KCut, but it need not yet be drain-closed. Core.Drain and Core.SettleKBridge allow only finite, non-replenishing cleanup and connect a persisting blocked frontier to the later drain-closed KCut at which Dead_K is evaluated. The finiteness premise comes from Core.16(6)/K.Drain and the qualified cycle/progress evidence, not from finite channel capacity by itself. Core.16's prohibition on source-later 'launch' is intentionally broader than message Issue: it also covers preparation/reach or other later work that could replenish the blocked component.
- **Supported alternatives.** Use automatic-flush pipelining or an equivalent proved no-new-later-work discipline. A pipeline or custom controller that continues launching source-later work - including preparation/reach that can replenish the blocked component, not only endpoint Issue - requires a different liveness argument and cannot rely on the ordinary drain-based reduction.

Scheduler robustness and the actual capacity configuration. ***(Appendix F: deadlock items 5-6)***

- **Design implication.** Do not make deadlock freedom depend on favorable overlap, same-cycle issue, or a preferred arbitration choice when claiming scheduler-robust A5-L. Check the selected design against its actual back-annotated capacities and explicit elaborated boundaries, not an idealized unbounded network or a different rendezvous/buffered configuration.
- **Why the proof cares.** Policy-S intentionally serializes process-facing blocking issue. K.OpKey supplies common typed occurrence keys. K.CVPred/K.CV-WF now do more than conditional value determinacy: the serial-route comparison-availability closure must make every required predecessor fact available on the liberal side without changing DomEpoch, releasing the frozen component, or introducing an unmatched dominance-domain endpoint transfer; recursively materialized non-transfer predecessors carry an explicit Core.16 reach-legality disposition. K.CompareAdvanceLegal separately covers the main P0 preparation/launch and R Issue/assertion compatibility. Ordinary or comparability-

certified singleton frontiers use K.CompareAdvanceLegal-Ord; explicit Sys_B^elab multi-frontiers require checked elaboration/order evidence. K.2b then proves that an A5-L counterexample would force a finite response failure on the selected conservative run.

- **Supported alternatives.** Rewrite or buffer the protocol, add explicit synchronization, change the selected capacity/elaboration and rebind the theorem instance, or use a scheduler-specific structural, invariant, exploration, or tool-certificate proof of A5-L.

Reset, environment, termination, and other excluded stalls. ***(Appendix F: deadlock items 7-8)***

- **Design implication.** Treat dynamic reset during the serial comparison through a certificate-supplied K.DomResetScope and the K.ResetEpoch gate; disjoint RW5 resets may remain legal. Treat modeled EOF/done/abort/end-of-test through the nested K.TerminalDisposition gate, not by an external host watchdog. Distinguish environment-only stalls and timed/reactive multi-frontier behavior from the internal wait-cycle predicate.
- **Why the proof cares.** Appendix K excludes a specific closed internal message-passing wait structure; it does not convert every form of nonprogress into the same graph predicate. An affecting reset can clear the exact pending operation, a declared terminal disposition can end the selected execution with it pending, an external event may release the component, and starvation or perpetual under-supply can persist without satisfying Dead_K. The Policy-S proof therefore evaluates Reset first, Terminal second, and enters the frozen infinite-continuation argument only on both Continue branches.
- **Supported alternatives.** Use the independently certified ResetEpoch.DirectFail or TerminalDisposition.DirectFail branch where applicable; use the uniform U1-U3 instance-level Continue discharge when one conservative scope can exclude all relevant resets/terminal pending frontiers; represent terminal/environment protocols through ordinary observable actions; remain within StandardEnv with the required K.JointFreeze evidence for admitted multi-frontiers, or within the certified single-internal-frontier restriction for timed/reactive/nondefault environments; otherwise discharge A5-L by a separate proof.

3.4 How to use Appendix F with this guide

The parenthetical tags on the eight Appendix-F-tagged themes give the complete item-level mapping to the nine safety items and eight deadlock items in Appendix F, preserving Appendix F as the checklist. Use Appendix F first as the architect-facing design-pattern screen. For each applicable item, identify the proof layer and evidence family described above, select the intended theorem scope (safety only or safety plus internal-deadlock preservation), and ensure that the tool/verification flow records the corresponding discharge in Appendix M.7a. This guide explains the dependency; it does not replace the normative constraint or the per-instance evidence record.

4. Core vocabulary and proof boundaries

PURPOSE	**Define the small set of objects that repeatedly connect the Normative Formal Core and Appendices G, I, J, K, and L. Engineer takeaway:** keep four distinctions visible: observable events versus generic epsilon micro-steps; delta-settling versus silent tau-cycles versus B6 idle ticks; committed history versus current residual offers; and safety correspondence versus liveness assumptions.

Term	Engineer-oriented meaning
Sigma (Σ)	The selected alphabet of observable events: committed Push/Pop transfers, synchronization events, and selected signal Read/Write events. The proof boundary determines which internal or DUT-boundary signals/channels are included.

Term	Engineer-oriented meaning
Epsilon (ε)	The generic proof-hidden micro-step relation. It includes source evaluation, failed non-blocking choices, and other internal behavior according to the model. It is not a synonym for a silent clock cycle. Delta-settling is a specific same-cycle settling classification; B5 may charge a silent tau-cycle to epsilon work without identifying the two.
δ-settling (delta)	Cycle-local same-cycle settling. On a phase-bearing source model, δ_ref is exactly the Core.Settle relation ->ε,L2; on RTL_B in the standard QSync scope, δ_post is the same-cycle settling relation of Core.QSync. Delta steps preserve cycle index and registered state. Other epsilon steps may also preserve the cycle but are not delta-settling.
Silent τ-cycle (tau)	A real clock cycle in which no selected Σ event commits while hidden registered/internal progress may occur. Tau is explanatory cycle-level terminology, not a transition label and not an epsilon step. B5 may prove that an internally advancing silent cycle consumes bounded epsilon work.
B6 idle tick	A real clock tick after the complete system has reached the B5/global fixed point. It performs no epsilon-prefix/suffix work and leaves non-clock state unchanged; it is treated as epsilon-stutter only by the observable trace-matching convention.
Observable unit	A singleton Σ event or an explicitly atomic same-cycle group such as a rendezvous Push/Pop, paired synchronization transaction, R2C fixed-point unit, signal-anchor group, or an explicitly selected proof-internal RESET unit. RESET may be carried by the Core.17 observable-unit machinery without being added to Σ.
Commit	Core.8 defines a boundary commit as the selected proof model's legal E4 channel-commit transition, producing the committed Push/Pop unit and abstract channel-state update. For RTL_B, boundary mapping/B-faithfulness relates that abstract event to the clock cycle in which mapped valid and ready are simultaneously satisfied and the payload is observed. Commit(q) is the designated process-facing completion/accept/release occurrence for source operation q; CommitCycle(q) is its cycle index. Commit/release is distinct from L4/Core.RegSeq returned-result availability.
Issue	The cycle attributed to a dynamic source message-operation instance when its endpoint-owned request begins. Function-like Issue(q) notation is shorthand for the Appendix-C relational existence/uniqueness facts. Continued-request attribution may span inserted latency; Issue is operational evidence and is not generally reconstructible from committed labels alone.
ReturnedAtCycle	The cycle boundary at which a message-call result becomes process-visible. For a successful blocking or non-blocking call it is the transfer-commit cycle; for a failed PushNB/PopNB it is the cycle in which the failure status returns even though no message commits. It is an operational timestamp, not a KObs field.
Sys_ref	The prescribed source-reference transition system: Sys_B in the ordinary back-annotated case, or Sys_B^elab when explicit staged, bundle, join, guard, coupler, or epoch-gate boundaries are required. Raw rendezvous Sys remains Appendix G's conceptual presentation; the G-K proof stack uses Sys_ref, with B(c)=0 recovering rendezvous semantics.
Horizon	The selected cycle/decision horizon used by J to group construction actions and enforce registered result availability and L1/L2/L3 phase ordering.

Term	Engineer-oriented meaning
Finite ideal	A finite down-closed set of observable units: if it contains a unit, it also contains every unit required to precede it. Appendix J constructs the source witness through a chain of these legal partial-order prefixes.
KObs	The compact Core.18c source cutpoint record <E,w,H,O>. Core.18 fixes the common record type and typed equality used for cross-model correspondence; Appendix J constructs the RTL-facing KObsPost_J representation and proves equality through J.KObsConf.
Residual offer	A typed current outstanding source-level request at an explicit abstract boundary, including process, frontier item, dynamic call, direction, and reset epoch.
WFG	Core.WFG is the wait-for graph over internal processes. For each disabled message frontier item x of P, an edge P -> Q points to the unique complementary endpoint owner Q. An edge alone does not make P blocked: P is blocked only when its frontier is nonempty and every frontier item is disabled, which matters for certified multi-frontier elaborations.
SettlePoint / BoundaryEvalPoint	SettlePoint is the first fixed point reached by the complete source L2 epsilon-settling relation after L1 issue closes and requests freeze; Definition Core.Settle itself is conditional on that endpoint existing. BoundaryEvalPoint is the settled explicit-boundary request/enablement interface. In the standard scope Core.QSync-Ref proves source SettlePoint existence and uniqueness.
Core.QSync	The qualified synchronous same-cycle model. QS1-QS5 require one global clock, registered/frame separation with fixed cycle inputs and stable sequential drives, deterministic local evaluation, a finite acyclic dependency graph, explicit-boundary completeness, and QS3 realization/adequacy tying allowed same-cycle transitions to the settling network. For RTL_B in the standard K-facing scope, QS2a additionally requires unique recovery of the registered/input frame and reachable K-origin for every reachable post KCut. Determinism is unique settled result for fixed entry/drives, not one global Sys_ref execution.
Core.KCut	The K-facing comparison domain: a reachable state that is model-specific epsilon-quiescent and whose explicit BoundaryReq/ChannelEnabled/ChannelDisabled network is at its selected fixed point. Source SettlePoints are KCuts; RTL has its own KCut domain and no source L0-L4 phase tag.
Core.KCutClosure_post	The downstream RTL KCut-existence/non-vacuity interface. In the standard QSync scope Core.QSync-Post derives it, while QS2a plus Core.QSync-KCutCanonical identify every reachable post KCut with the unique canonical endpoint of its recovered same-cycle K-origin and supply the canonical NormPost witness. K.CertCov remains a distinct package-totality interface, but the preferred standard route derives it from J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync.
Core.SyncFence	Constitutive Sys_ref synchronization-completion legality: a synchronization may commit only after every earlier blocking Push/Pop in pref_S(s) has process-facing committed, possibly in the same cycle. On RTL, the same rule is a conformance obligation; equal-cycle “before interval advance” accounting is certificate attribution, not observable physical sub-cycle ordering.
Core.MemSyncOrder	Local schedule/mapping-layer qualification for array or memory accesses source-ordered with synchronization calls: mapped storage commits before s occur no later than s; commits

Term	Engineer-oriented meaning
	after s occur strictly later. It is recorded in M.7a but does not add memory events to Σ or become an Appendix I/Theorem J.1 replay premise; J.Mem remains separate for cross-process shared-memory value flow.
Core.ReachPrep / Core.IC	ReachPrep permits finite dependency-independent L1 preparation/reach of a source-later operation before an earlier blocking result returns, without erasing source order or implying Issue. Core.IC is the constitutive Sys_ref Issue/Commit lifecycle invariant: after Issue, the same attributed blocking request persists across inserted latency until process-facing commit or affecting RW2 RESET; DeclaredTerminal may end execution while it remains pending. The corresponding RTL lifecycle/attribution behavior is a separately checked M.7a conformance obligation.
Core.CycleResult / Core.CycleClosure	CycleResult is the typed real-cycle object carrying full next state, committed observable units, and scoped RESET units. Core.RTLCycleCarrier derives a narrow post-side fact from this carrier: a reachable prefix that has already entered a legal RTL cycle and contains a complete selected unit can be carried through that same cycle to the successor K-origin; this is not generic RTL progress from an arbitrary K-origin. Core.CycleClosure separately qualifies legal nonterminal source cycles and the partial-cycle extension used by Core.CycleChoiceExt; when K.2b.E builds a fair continuation, Core.FairPick/FairCycleDovetail supply the constructive selector/dovetail bridge.
Core.IssueFair / Core.TerminalIdle	IssueFair is weak fairness for continuously legal source Issue opportunities and is separate from B2/B3 commit fairness. Core.TerminalIdle is the stronger machine+environment no-future-enabling condition required for a B6 idle suffix; an ordinary temporarily silent cycle is a Core.SilentCycle.

DETERMINISM TAXONOMY - do not conflate these notions		
Notion	**Logical status**	**Engineer-oriented meaning**
Same-cycle settling determinism	Model-class restriction	R2C/Core.QSync: fixed registered state, fixed cycle-t inputs, and fixed explicit request/issue drives yield one settled boundary valuation.
Conditional synchronous-step determinism	Functional cycle view after explicit choices are fixed	Once the currently legal witness, issue/scheduling, arbitration, and environment choices are fixed, the cycle can be viewed functionally. This does not remove Policy-L latitude or WC-sensitive source nondeterminism.
B1 observable RTL trace determinism	Theorem-instance assumption	Under fixed stimulus and initial state, RTL_B has one observable Sigma-projected committed history; internal epsilon behavior may still differ.
I.DR / I.DR’	Derived theorem	Replay-checkpoint equality proved from the Appendix I premises. It is not an independent assumption to enter in the M.7a discharge record.
K.CV	Appendix K premise	Appendix K's serial-route value/identity determinism uses a common CVFact key universe, finite

		Pred_CV sets, deterministic per-fact evaluators, explicit predecessor occurrence closure, and K.CV-WF (or a decreasing rank). The same certificate also carries the comparison-availability closure used by K.2b.P0/R/P and an explicit Core.16 reach-legality disposition for recursively materialized non-transfer predecessors. The broad Appendix-G causal graph and rendezvous mutual enablement are not automatically provenance predecessors.
SDet	Selected-instance fixing/premise	Fixes the Policy-S simulator, scheduler, arbitration, normalization, and related choices so the selected instance has no second incompatible committed history.
Normalization selection/congruence	Selected proof data plus theorem	Records which replay normalization is selected and why the replay quotient is preserved. Core.QSync canonicalizes only K-facing same-cycle settling; Appendix I/J replay normalization remains separate.

For theorem premises, Appendix M.7a supplies the evidence axis: inapplicable to the selected theorem scope, satisfied by a syntactic/design restriction, discharged by tool/RTL evidence, or discharged by a source/RTL witness construction. A required condition may not be marked inapplicable; derived theorems such as I.DR do not become independent premises merely because their proof artifacts are recorded.

CUTPOINT BOUNDARY: KObs is not full-state equivalence

KObs is the typed Core.18 cutpoint record <E,w,H,O>. Core defines the source record, its pure reconstruction functions, and the cross-model equality notion, but deliberately does not define how raw RTL state yields the record. J.OfferReach proves H/O is realized at the selected reachable source KCut; J.OfferConf audits O bidirectionally against the complete KObs-relevant RTL request inventory; J defines KObsPost_J(rho_post;E,w,H,O) and J.KObsConf proves it equals KObs_E,w(sigma_ref) for the exact selected package. Core.SyncFence ensures source residual blocking offers belong to the current interval. Core.18d/e then reconstruct process/channel/frontier/request/enablement/WFG/snapshot-drain facts from the common record, while Appendix K separately factors deadlock, waiver, environment-terminal, and diagnostics. Equal KObs is therefore typed observation equality, not full-state equivalence or equality of historical Issue timelines.

4.1 Safety direction: why Post -> Ref is the normal signoff argument

For ordinary safety signoff, the useful deductive direction is: every covered reachable RTL prefix has a matching reachable Sys_ref prefix with the same canonical observable history and Core.17 ΣMatch events/values. Therefore, if all relevant Sys_ref histories satisfy a J.HCanon-invariant safety property meeting Definition J.IdealPrefixClosed, the covered RTL histories do too. Here “prefix-closed” means downward closure over finite required-order ideals, not merely deleting a suffix from one chosen linearization of independent events. When one complete source run is used as the verification representative, J.RefProjCover must cover the relevant source observations by finite canonical ideals/down-closed observable prefixes of that run. In the standard QSync route, J.CycleReach_post/J.QSyncTraceInd/J.TraceCertCov-QSyncCycle first establish the cycle-aligned Post -> Ref packages; Core.RTLCycleCarrier and J.CycleIdealCover then cover every reachable finite outer observable history by a reachable cycle-aligned history, and J.1-Safe-QSync uses J.IdealPrefixClosed to obtain the all-finite-prefix safety conclusion without creating a separate J.GWR/J.TraceCertCov package for each intra-cycle ideal. Assertion-style no-bad-event invariants, per-channel FIFO/value legality, and explicitly downward-closed protocol invariants are natural uses. A cross-interface scoreboard whose check depends on an otherwise droppable explanatory event may fail J.IdealPrefixClosed and should instead use direct J.TraceCertCov on the required arbitrary-prefix domain (or refine the retained observation/order).

The reverse direction is intentionally restricted. J.RTLConsistentRefPrefix is typed over a reachable source prefix already equipped with a selected reachable RTL prefix and compatible annotations. Their complete selected observable-unit projections must match under Core.17 - neither side may contain an extra unmatched selected unit - and any cutpoint use must carry the matching J.OfferReach/J.OfferConf/J.KObsConf package. Because the RTL witness is already supplied, the Ref -> Post clause does not depend on a generic fair-extension theorem. Appendix L can select such a source witness when latency-sensitive choices require RTL-aligned epsilon outcomes.

5. Normative Formal Core and Appendix G - shared semantics and observable contract

PURPOSE	**Define the shared source-reference semantics and the observable scheduling contract that the post-HLS implementation must preserve.** **Engineer takeaway:** The Core is the normative semantic skeleton for G-K; Appendix G constrains that skeleton through R1-R5 and E1-E5. Neither layer by itself constructs the matching source execution - Appendices I and J do that work.

5.1 Source-side rules R1-R5

Rule	Role in the architecture
R1 - synchronization order	Synchronization calls of each process remain in dynamic source order and commit in strictly increasing cycles.
R2 - signal anchoring	For a sequential process, a signal Read is logically anchored to the closest preceding synchronization; a signal Write is anchored to the closest succeeding synchronization.
R2C - combinational fixed point	Combinational observations are defined from the cycle-entry valuation and the unique settled result of the qualified component/network. Internal epsilon/stutter re-evaluation is permitted so long as it converges to that unique result. R2C fixed-point units are matched by component identity and local emitted-occurrence ordinal rather than by a global cycle bucket; Sys_ref and RTL_B need not share an internal combinational model or absolute cycle numbering. ComponentParticipatesInCycle defines each side's observation domain. A cycle-locked component emits exactly one complete fixed-point unit in every participating cycle and none outside it; per-side participation/completeness and the implementation/certificate justification are part of the R2C conformance evidence.

Rule	Role in the architecture
R3 - sync isolation	A synchronization boundary separates the message operations in the immediately preceding and succeeding source intervals, and its Core.SyncFence completion clause requires every earlier blocking Push/Pop in pref_S(s) to process-facing commit no later than the synchronization commit. Same-cycle completion is permitted.
R4 - safe issue order	Dynamic message operations preserve source issue order with the general non-strict prefix-closed inequalities. The stricter special case applies only to distinct blocking instances on one scalar A9-governed explicit boundary/direction: a later same-endpoint blocking instance may not begin while the prior distinct blocking instance remains the attributed outstanding request. Continued attribution may span inserted latency.
R5 - channel capacity semantics	The source reference uses rendezvous or finite-capacity channel semantics at the selected explicit abstract boundaries; elaboration may introduce proof-internal channels but does not reorder ordinary source calls.

DEFINITION: Core.RegSeq - registered sequential-interface semantics

For a sequential process, request and Push-data outputs in cycle t depend only on registered state and operands fixed before L2 settle. If ReturnedAtCycle(q,t) holds - for a success or failed non-blocking return - any dependent evaluation, control, endpoint/address, payload, or issue has a later Horizon. ReturnedAtCycle is defined normatively in Appendix C; Core.RegSeq states the dependent-use rule.

- **Why it matters.** It eliminates same-Horizon decision-to-dependent-request edges that would be inconsistent with clocked hardware.
- **Where it is used.** Appendix I local preparation, J.FoundationRank/J.DepRank, J.RegPhaseNF, and the Appendix K serial-reduction assumptions.
- **DV implication.** Checked source/tool evidence must establish the registered-cycle rule. Zero-time dependent NB cascades must be identified and then rejected, excluded, isolated, remodeled atomically, or aligned through Appendix L; silent scope exclusion is forbidden.

CLARIFICATION: Core.OrdFrontSingleton - ordinary-source frontier uniqueness

Within an ordinary non-elaborated sequential source interval, dynamic message calls remain ordered by dynamic program order, including distinct-interface calls, repeated loop iterations, and ordered unrolled instances. Therefore Front_P has at most one member. A genuine message-operation multi-frontier requires explicitly modeled incomparable Sys_B^elab frontier items with the corresponding order, boundary, projection, and attribution evidence.

5.2 Normative Core operational semantics and the cycle-level hardware model

The Normative Formal Core treats Sys_ref as an operational transition system with explicit process state, explicit abstract channel state, theorem-instance dynamic operations, pending requests, and epsilon micro-state. Core.8 defines a boundary commit as the selected proof model's legal E4 channel-commit transition; on RTL_B, boundary mapping and B-faithfulness/conformance relate that abstract event to the cycle in which the mapped valid/ready conditions and payload are observed. Commit(q) is separately the process-facing commit/accept/release occurrence designated as completing the source operation, and CommitCycle(q) is its cycle index; completion does not make returned data/status available for same-cycle dependent use, which remains an L4/Core.RegSeq fact. The Core separates dependency-independent L1 ReachPrep from actual Issue and requires prefix-closed issue attribution. Definition Core.IC is the exhaustive ordinary blocking-request lifecycle invariant and Definition Core.SyncFence is source synchronization-completion legality; both are constitutive Sys_ref semantics, while their mapped RTL behavior is a named implementation-side conformance obligation. For complete/fair source-continuation proofs, Core.CycleState/Core.CycleResult carry the full registered, environment, clock, reset-unit, and cycle-valued monitor state; Core.CycleClosure qualifies legal complete source cycles and partial-cycle extension. Core.RTLCycleCarrier is a separate narrow RTL typing/completion lemma: once a reachable prefix has already entered a legal real cycle and contains a complete selected unit, the enclosing CycleResult carries that same execution through the already-entered cycle to its successor K-origin. It is not generic cross-cycle progress from an arbitrary K-origin. Core.IssueFair/B2/B3 remain the behavioral fairness/progress properties, while Core.FairPick/Core.FairCycleDovetail are the stronger constructive evidence form used when Appendix K must build such a continuation.

Policy	Meaning and use
Core.PolicyL / Core.LLegal	Core.PolicyL is the permissive issue policy used by the safety witness and A5-L. Core.LLegal fixes exact step legality. Core.ReachPrep may allow a later dependency-independent operation to become reached before an earlier same-interval blocking call commits; reach is not Issue, source order remains, and Post -> Ref coverage must ensure Sys_ref is permissive enough for every preparation segment used by the accepted J construction. QSync does not remove this issue latitude.
Core.Phase / Core.LAdm	Core.Phase defines L0-L4 and the pre-settle/settled-frontier boundary. Core.LAdm qualifies finite Policy-L prefixes and complete executions by adding the drain/settle-to-KCut discipline. When K.2b.E constructs a complete fair continuation it additionally consumes Core.CycleClosure/Core.CycleChoiceExt and Core.IssueFair; temporarily silent cycles are real CompleteCycle/SilentCycle transitions, while a B6 suffix requires Core.TerminalIdle.
Core.PolicyS	The conservative serial-blocking issue policy: a later same-interval blocking operation may issue only after every earlier blocking operation has process-facing committed in a strictly earlier cycle. It selects the instrumented deadlock-stress run used by the primary Appendix K route.

DEFINITIONS: Core.Phase, Core.Settle, Core.KCut, Core.16, Core.Drain, and Core.LAdm
Core.Phase gives each source cycle a process-owned L1 preparation/issue window, L2 settling with requests and selected cycle-t inputs frozen, L3 boundary actions, and L4 registered result availability. Core.ReachPrep governs dependency-independent eager preparation within L1; reach and Issue remain distinct. Definition Core.Settle names the first L2 fixed point if reached, and Core.QSync-Ref proves finite existence/uniqueness in the standard scope. Core.SettleIsKCut makes it a source KCut. Core.16 activates when the minimal frontier is fully disabled and forbids source-later launch broadly enough to include replenishing preparation/reach, not only message Issue. Core.Drain permits finite non-replenishing cleanup; its finiteness comes from Core.16(6)/K.Drain plus the stated progress/qualification evidence rather than capacity alone. Core.SettleKBridge connects a persisting activation to the later drain-closed KCut. For complete continuations, Core.CycleResult/Core.CycleClosure type the real cycles and Core.CycleChoiceExt extends selected phase-legal partial cycles; Core.IssueFair is separate weak fairness. RTL_B has no source phase tags; Core.QSync-Post supplies its settled KCut/closure interface.

ARCHITECT MENTAL MODEL - one ordinary synchronous cycle	
Cycle-level view	**Formal correspondence**
1. Registered state and selected cycle-t exogenous inputs are fixed.	QS1/QS2; ClockScope audit; L2/QSync frame.
2. Sequential process request/payload outputs are determined from registered state and operands already fixed for the cycle.	Core.RegSeq. Ordinary combinational logic from registers is allowed; a newly returned cycle-t result cannot feed a dependent same-cycle sequential output.
3. The complete system combinational/ready-valid/guard/join/coupler network settles to one deterministic valuation.	Delta-settling; R2C; QS3/QS4; Appendix Q for Sys_B^elab. Pure combinational loops are outside the standard scope; feedback crosses explicit state.
4. The settled boundary snapshot is read.	BoundaryEvalPoint; source SettlePoint; Core.KCut when the K-facing proof consumes the snapshot. These are proof views of the ordinary settled cycle state, not additional hardware states.
5. Boundary decisions and commits occur.	Core.Phase L3; E4; R3/E5; Core.SyncFence.
6. The next registered state/result availability is established.	Core.Phase L4 / next cycle. RTL_B has its own registered update rather than source L0-L4 tags.

The functional cycle picture is conditional on explicit choices, not a claim that Sys_ref has only one execution. WC-sensitive outcomes, Policy-L issue/scheduling choices, arbitration, and reactive-environment choices remain part of the admissible execution space. Likewise R2C/Core.QSync determinism means a unique settled result for a fixed cycle-entry valuation and fixed drives/choices; internal epsilon/stutter re-evaluation is allowed, and Sys_ref and RTL_B need not share the same internal combinational model or absolute cycle buckets.

5.3 Post-HLS compatibility obligations E1-E5

Obligation	What the RTL/tool flow must establish
E1	Preserve each process's synchronization-event order.
E2	Preserve anchored signal values/visibility for sequential processes and fixed-point signal units for combinational processes.
E3	Preserve safe message Issue order and attribution, including same-interface order, distinct-interface no-reverse order, and the prefix-closed Issue condition.

Obligation	What the RTL/tool flow must establish
E4	Realize legal rendezvous or finite-capacity FIFO behavior at each selected explicit abstract channel boundary, including payload/order/occupancy behavior.
E5	Do not move a message operation across its immediate synchronization boundary: predecessor operations issue no later than the sync; successor operations issue strictly later; and every earlier blocking Push/Pop in pref_S(s) must process-facing commit no later than the sync. Same-cycle message/sync completion is allowed. On the post side, the "accounted before interval advance" convention is certificate/trace attribution, not a physical sub-cycle RTL ordering requirement.

BOUNDARY CONDITION: B-faithfulness, Core.ElabProj, and Sys_B^elab

E4 only has meaning at a chosen abstract boundary. B-faithfulness must show that every transfer crosses that boundary and that occupancy/enablement match B(c). Core.ElabProj supplies four distinct roles when the implementation is elaborated: observable trace projection, occupancy projection, conservation/accounting, and proof-internal WFG visibility. Internal movement remains epsilon-opaque at the selected visible boundary, but any hidden grant, throttle, join, coupler, or epoch-gate condition that changes availability must be exposed through Sys_B^elab and its explicit boundaries.

5.4 Trace compatibility is observational, not temporal equivalence

The relation approximately-equal (≈) is Core.17 ΣMatch plus E1-E5. ΣMatch explicitly matches the complete selected occurrence sequences and value labels - including per-channel Push/Pop ordinals, canonicalized sequential signal observations, R2C fixed-point units matched by component/local emitted-occurrence ordinal, synchronization occurrences, and any explicitly selected proof-internal RESET units carried by the observable-unit machinery (without adding RESET to Σ). Independent matched observations may occur in different cycles or be grouped differently where the ordering and atomicity rules permit. Appendix G expands this matching convention; it does not define a second competing trace relation.

Appendix G's explanatory causal-dependency graph is not a universal well-founded proof order. It may contain legal same-cycle atomic cycles. Appendix J instead generates its own construction relation: D1-D4 are semantic prerequisite edges covered by J.DepSound, while D5 is Horizon proof stratification and is justified separately by J.HorizonOrderSafe commutation evidence. Appendix K uses yet another relation for Policy-S value/identity determinacy: K.CVPred/K.CV-WF is a well-founded fact-provenance order and must not inherit cycles merely because two operations are causally or temporally adjacent.

READING RULE: What to take from Appendix G

For architects: the Core fixes the operational model and G defines what implementation freedom remains invisible to a latency-insensitive environment. For DV: together they identify the observable checker boundary and conformance obligations. For formal implementers: the Core supplies the shared transition-system/KObs/WFG/drain vocabulary, while G supplies the R/E contract consumed by I and J.

6. Appendix I - per-process replay and preparation

PURPOSE	**Show that each process can locally replay the selected RTL observable prefix and prepare for its next participation using a finite source-owned script.** **Engineer takeaway:** Appendix I is intentionally local. It identifies the right dynamic operation and reaches its request/decision point, but it does not assert that the complementary endpoint is present or that all process scripts can be merged globally.

6.1 Inputs and preconditions

Input	Why I needs it
Fixed initial state and stimulus	The process is replayed under the same global Sigma-causal environment behavior, not an arbitrary P-local stimulus.
WC witness	Every epsilon-modeled choice that can affect later observable control, values, identities, requests, or frontiers is fixed consistently.
I.A0 / E3/E5	The RTL message-issue discipline is supplied as an implementation assumption, including prefix-closed issue attribution and the E5/Core.SyncFence rule that no earlier blocking request survives a synchronization commit into the next interval.
Core.RegSeq	Dependent next-cycle requests cannot be created from same-cycle returned results.
B4 / finite pipeline depth	Internal micro-progress has a finite epsilon bound when the process is not externally stalled.
B2/B3 when progress is invoked	Continuously enabled actions and complementary channel discharge are eventually selected; these are environment/scheduler assumptions, not consequences of source code.
Source well-formedness	Signal anchors, dynamic operation identities, direct-input contracts, shared-memory lowering, and reset behavior are well formed.

6.2 The replay checkpoint: I.ReplayObs, I.CutpointOfferObs, I.ReplayObs-Congruence, I.DR, and I.DR’

I.ReplayObs is the logical P-local replay checkpoint determined by the committed observable-unit history, fixed stimulus, and witness. It contains the logical replay control position, replay-relevant values, witness-selected outcomes, replay-fixed dynamic attribution, completed-item status, candidate NextFront_P, anchored values, returned results, reset epoch, and normal-completion status. It is not the complete raw operational state: certified outcome-independent L1 preparation may advance the operational cursor or compute temporaries without changing the logical checkpoint.

I.CutpointOfferObs is the separate operational slice for current uncommitted blocking offers: Pending_P, Front_P, BoundaryReq, owning dynamic calls, boundary/direction/epoch facts, and stable Push payload obligations. The K-facing cutpoint therefore combines replay history H with residual offers O. Deterministic replay does not manufacture O from H; Appendix J constructs and audits O through J.OfferReach and J.OfferConf.

LEMMA: I.ReplayObs-Congruence
From equal logical replay checkpoints, the same next complete observable unit produces equal canonical immediate post-unit checkpoints. An exact selected finite normalization may then mix replay-inert preparation segments, which leave each checkpoint unchanged, with witness-fixed epsilon transitions that may update replay fields. The normalization convention must pair those replay-changing transitions so that equal checkpoints remain equal; the theorem does not assume per-path neutrality, global confluence, or uniqueness of all legal raw endpoints.

LEMMA: I.DR and I.DR' - deterministic replay

I.DR' establishes equality of the logical ReplayObs checkpoint at the canonical immediate post-unit state, before optional following L1 preparation or epsilon-normalization. I.DR establishes equality at the selected epsilon-quiescent cutpoints by applying paired congruence to the exact proof-relevant normalization paths. Appendix J uses the immediate form when constructing the next Horizon and records normalized witnesses explicitly when a quiescent cutpoint is required.

6.3 Deferred non-blocking decisions

A process-local proof cannot decide a non-blocking poll from process state alone because success or failure depends on the common incident-channel snapshot and possible complementary requests. Stopping the local replay at every poll would be too restrictive: a registered process may execute independent same-Horizon work that does not depend on the poll result.

DEFINITION: I.DeferredNBHole

A deferred hole records the dynamic non-blocking call, explicit boundary, snapshot key, Horizon, result destinations, reset epoch, witness-selected expected outcome, and NoDependentUse evidence. The process-local preparation script can carry the unresolved hole through independent work while forbidding any dependent use of its result.

6.4 The local preparation theorem

LEMMA: I.3 - finite process-local preparation script

Given the replay state, exact dynamic operation identity, Core.RegSeq, WC, E3/I.A0, source well-formedness, and typed footprints, I.3 constructs a finite process-owned epsilon/request path to the selected operation, its persistent issued request, or a carried deferred non-blocking decision hole. The script does not commit an unmatched observable unit and does not assume global enablement.

- **Output.** The information later packaged in J.PCert: the logical ReplayObs checkpoint, exact dynamic identity, finite per-Horizon preparation actions, separate operational issue/pending attribution, stable payload/value facts, field-sensitive footprints, and deferred-hole records.
- **Non-result.** No complementary endpoint, common channel snapshot, globally legal merge, or reachable system extension is proved here.
- **Why finite.** The source-control segment is finite and internal progress is bounded by B4/FPD when not externally stalled.

6.5 Engineer example: independent work after a poll

Suppose a process performs a PopNB, updates an independent diagnostic counter, prepares a request on another endpoint whose control and payload do not depend on the PopNB result, and only later branches on the poll status. Appendix I may carry the poll as a DeferredNBHole through the independent counter/request work. Appendix J later freezes the complete channel request snapshot at L2, decides the poll once globally, and prevents the branch from occurring until the result is available under Core.RegSeq.

ARCHITECTURAL SEAM: Why Appendix I cannot be the whole safety proof

Local scripts may each be valid when viewed against read-only summaries, yet still conflict through shared channel state, atomic rendezvous pairing, a common non-blocking snapshot, reset scope, or overlapping field accesses. Appendix J proves that the scripts coexist and derives a legal global order; it does not merely conjoin them.

7. Appendix J - local-to-global safety construction

PURPOSE	**Construct one reachable Sys_ref execution from local process/channel/atomic certificates and prove observable compatibility with a selected RTL prefix.** **Engineer takeaway:** Appendix J is the main safety theorem. Its defining achievement is that global witness realizability is derived from local evidence rather than accepted as an unexplained system-level premise.

7.1 Theorem J.1: the top-level safety result

THEOREM: J.1 - execution-level / trace-set compositional compatibility

For every covered reachable RTL_B prefix, the normal compositional route uses Definition J.LocalGWR, QualifiedSynchronousModel_Sys_ref(I)/Core.QSync-Ref, and Theorem J.GWR-Comp to construct a reachable Sys_ref prefix and a global witness package whose annotated projections satisfy E1-E5. A directly validated J.GWR package is an alternate route and may carry equivalent checked per-Horizon L2-settling evidence instead of the global source QSync premise. Plain finite-prefix J.1 does not require post-side QSync unless a post KCut/NormPost interface is invoked. The reverse Ref -> Post clause applies only to source prefixes already equipped with an RTL-consistent annotation/witness package: that premise already selects tau_post*; theorem-wide B-faithful/E4 facts supply Proposition J.0's channel premises; J.0 performs the conditional composition; and the existential post witness is tau_post := tau_post*.

Four details are important for engineering use:

- **Finite reachability, not only fair executions.** The generic Appendix I/J domains are Reach_post and Reach_ref finite reachable prefixes. Theorem J.1 does not require generic Exec_post/Exec_ref objects or a J.FE fair-extension lemma. Complete/fair source continuation is introduced only by later liveness machinery such as Core.LAdm/Exec_L^adm and K.2b.E.
- **Required-prefix induction.** The proof orders observable units by the constraints that must be preserved, rather than by raw RTL cycle buckets or full source order across independent endpoints.
- **Annotated carrier.** Compatibility includes dynamic identities, witness choices, Issue/Commit attribution, request facts, and channel histories. It is stronger than matching an unannotated list of labels, but weaker than full-state equivalence.
  - **Source-side settling route.** On the normal J.LocalGWR -> J.GWR-Comp route, QualifiedSynchronousModel_Sys_ref(I) and Core.QSync-Ref supply the finite L2 SettlePoint needed by J.RegPhaseNF/J.UnitExt for every constructed Horizon. A directly validated J.GWR package may instead carry independently checked finite L2-settled segments for its represented Horizons. Plain finite-prefix J.1 does not thereby require post-side QSync unless a post KCut/NormPost interface is also used.

7.2 The local certificate package

Component	What it certifies
J.PCert	Per-process logical replay checkpoint plus separate finite preparation/issue evidence, exact dynamic identity, field-sensitive footprints, and deferred non-blocking holes produced from Appendix I.
J.CCert	Per-explicit-channel boundary identity, capacity, E4 history, occupancy/head, payload, request snapshot, enablement, reset, and B-faithfulness evidence.
J.ACert	Agreement for explicitly atomic units: rendezvous pairs, paired synchronization, E2/R2C groups, RESET, and other named multi-participant actions.
J.EnvCert / J.ResetCert	Environment and reset-unit identity, scope, action, and locality evidence.

Component	What it certifies
Typed footprints	Field-sensitive ReadSet/WriteSet evidence for every construction action, including component-local elaboration actions.
LocalAgree	Equality, uniqueness, ownership, coverage, and duplicate-field agreement across local records.
Generated dependencies	Locally justified Dep_J edges and enough evidence to prove that omitted pairs commute; the package does not supply a convenient global rank.

DEFINITION: J.LocalGWR

The complete local package for one RTL prefix: PCert for every process, CCert for every explicit boundary, ACert for every atomic unit, applicable environment/reset records, complete action/deferred-hole inventory, field-sensitive footprints, Core.RegSeq and source conformance, generated dependencies, LocalAgree, and coverage. It must not assume that these records already form a realizable global replay.

7.3 How J derives a global construction order

J converts the certificate package into a finite graph of primitive construction actions. D1-D4 encode actual semantic prerequisites from process control/data/issue, shared snapshots, channel state, resets, environment, and explicit component rules; J.DepSound applies only to those semantic edges. D5 may additionally order cross-Horizon actions for proof stratification. A D5-only pair must carry J.HorizonOrderSafe evidence that the actions commute without changing legality, complete observable units, terminal post-settle semantic state, or the J.HCanon quotient. J.FoundationRank/J.DepRank then prove acyclicity without assuming the order that the construction is meant to derive.

Definition / lemma	Role
J.DepJ	Generated dependency relation with two roles: D1-D4 are semantic prerequisites (process control/data/Issue/ReturnedAtCycle, request/snapshot, channel/FIFO, reset/environment/elaboration); D5 is Horizon proof stratification and is not automatically an operational prerequisite.
J.DepSound / J.HorizonOrderSafe	J.DepSound proves only D1-D4 semantic edges are real operational prerequisites. For a D5-only edge with no semantic path, J.HorizonOrderSafe requires a commutation certificate showing the actions may be exchanged without changing legality, complete observable units, terminal post-settle semantic state, or J.HCanon.
J.DepComplete	If two non-atomic actions are left unordered, they either commute by field-sensitive framing or are the explicitly proved same-cycle buffered Push/Pop dual update. D5 may order already-commuting cross-Horizon pairs but cannot hide an actual semantic interference.
J.FoundationRank	A graph-independent lexicographic semantic rank based on reset epoch, Horizon, semantic stage, local ordinal, and component tie data.
J.DepRank	Every generated edge strictly advances the foundation rank.
J.DepAcyclic	A directed cycle would strictly increase the well-founded rank back to its start, which is impossible.
J.CanonicalRank	A derived deterministic totalization of the acyclic prerequisite relation. Stable tie keys order only commuting actions.

LOAD-BEARING RESULT: J.DepComplete + J.DepRank

The flow cannot simply provide a global rank that is assumed correct. It must generate every non-commuting prerequisite from local rules, prove that unordered actions really commute, and prove each edge advances an independent semantic foundation rank. This is what makes the construction auditable and prevents a hidden global replay assumption.

7.4 Registered phase normalization

Within each Horizon, the proof must respect actual clocked interface semantics. It cannot decide a non-blocking result and then create a dependent request in the same cycle, and it cannot evaluate different polls against inconsistent partial channel snapshots.

LEMMA: J.RegPhaseNF - registered phase normal form

Every finite same-Horizon construction fragment can be permuted, without changing its canonical observable history, into: L1 process evaluation and request/attempt writes; one finite L2 settle producing the common snapshot; and L3 non-blocking decisions plus complete boundary commits. The lemma needs the local fact that the frozen L2 state admits a finite ->ε,L2 path to a SettlePoint. On the normal compositional route Core.QSync-Ref supplies that fact uniformly; a direct J.GWR route may carry the equivalent checked per-Horizon evidence. Registered results become available only in L4/next cycle.

LEMMA: J.NBDecision

Once all L1 predecessors have executed and J.CCert supplies the common settled snapshot, each deferred non-blocking call has exactly one legal source result. A failed poll cannot coexist with a matching enabled rendezvous request, and a successful poll cannot be invented in a full/empty state that forbids it.

7.5 From one Horizon to the complete source witness

Lemma / theorem	Function in the construction
J.Frame	A certified transition changes only its WriteSet; disjoint state and guards are preserved.
J.LocalLift	Embeds one Appendix I process-local script into a reachable global state when owned/shared locations match its certified summaries.
J.PrepareCommute	Moves independent preparation actions into the common L1 window while respecting snapshots and true dependencies.
J.UnitEnable	After L1/L2, the requests, payloads, channel state, atomic participants, and holes make each selected L3 unit genuinely enabled.
J.UnitExt	Executes one complete Horizon bucket, updates replay/channel/atomic state, resolves holes, and preserves the construction invariant.
J.GWR-Comp	Outer induction over Horizon buckets constructs the complete reachable Sys_ref witness chain and the stable J.GWR interface.

THEOREM: J.GWR-Comp - local certificates construct global witness realizability

From J.LocalGWR plus QualifiedSynchronousModel_Sys_ref(I), Core.QSync-Ref supplies a finite source L2 SettlePoint for every reachable constructed Horizon. J.GWR-Comp then validates the dependency graph and construction order, phase-normalizes each Horizon, applies Horizon-batched J.UnitExt, and produces J.GWR and J.RegPhaseReach for one reachable Sys_ref witness.

Local certificates
↓
Generated dependency graph
↓
J.DepRank / J.DepAcyclic
↓
Validated order with finite-ideal prefixes
↓
Core.QSync-Ref + J.RegPhaseNF + Horizon-batched J.UnitExt
↓
J.GWR + J.RegPhaseReach
↓
Proposition J.0
↓
Theorem J.1

Non-circularity. LocalGWR supplies no global order, no phase-normal replay, and no global extension premise.

Stable interface. J.GWR can also be supplied directly by a translation validator or symbolic replay tool; that direct route must independently validate the same reachability, ordering, ideal-prefix, and finite per-Horizon L2-settling interface.

Reset handling. RESET is an atomic observable unit; RW1-RW5 and Lemma J.RS splice a new matched reset epoch while framing unaffected components.

Finite-ideal invariant. Every flattened prefix of the validated order is down-closed: if it contains a unit, it contains all required predecessors. J.UnitExt therefore grows the witness from one legal partial-order prefix to the next. Tie ordering among independent units is bookkeeping; required predecessors are not skipped and atomic units are never split.

7.6 Proposition J.0 is deliberately separate

PROPOSITION: J.0 - pairwise composition lemma

Given an already selected compatible source/RTL trace pair, compatible per-process annotated projections, and per-channel E4 well-formedness, J.0 lifts E1-E5 to the system level. It does not construct the source or RTL trace, prove B-faithfulness, or establish J.GWR. On J.1's normal Post -> Ref route, J.GWR-Comp constructs the reachable source/RTL pair and Theorem J.1 instantiates J.0 with that constructed pair. In the restricted Ref -> Post clause, RTLConsistentRefPrefix instead supplies the already-selected pair and J.0 is the non-circular composition step.

7.7 Safety transfer for the verification flow

DEFINITION: J.TraceCertCov + COROLLARY: J.1-Safe
J.TraceCertCov_I(D) is the named Post -> Ref safety-certificate coverage premise. For every RTL prefix in D it records either the normal J.LocalGWR/J.GWR-Comp package with source Core.QSync-Ref or a directly validated J.GWR package with equivalent checked per-Horizon settling, plus the remaining Post -> Ref premises. Corollary J.1-Safe consumes that route-sensitive coverage for a J.HCanon-invariant predicate satisfying J.IdealPrefixClosed. If one complete pre-HLS run is used as representative, J.RefProjCover must cover every relevant reachable source observation as a finite canonical ideal/down-closed prefix of that run. For D = CycleReach_post(I), the standard QSync route derives this package coverage from J.QSyncTraceInd. These safety-side coverage predicates do not imply K.CertCov.
QSYNC SAFETY PROFILE: J.CycleReach_post / J.QSyncTraceInd / J.CycleIdealCover / J.1-Safe-QSync
J.CycleReach_post is the exact reachable RTL-prefix domain ending at K-origins, before same-cycle QSync settling. J.QSyncTraceState retains only the exact-prefix LocalGWR/J.GWR-Comp witness and J.HCanon needed by safety. J.QSyncTraceInd proves a base case and one-cycle preservation for every reachable successor (not just one observed path); it explicitly does not consume J.OfferReach, J.OfferConf, J.KObsConf, K.DrainReach, or K.RLAdmReach. J.TraceCertCov-QSyncCycle and J.1-Safe-QSyncCycle establish cycle-aligned package coverage/safety. Core.RTLCycleCarrier supplies the execution-carrier fact for an already-entered RTL real cycle; J.CycleIdealCover then proves every reachable finite outer canonical history is a finite ideal of a reachable cycle-aligned history, and J.1-Safe-QSync uses J.IdealPrefixClosed to transfer safety to every reachable finite RTL prefix without asserting a separate J.TraceCertCov package for each intra-cycle ideal. If the richer K-facing J.QSyncCertInd is already established, J.QSyncCertInd-Trace supplies the safety induction automatically. Cross-interface scoreboards that fail J.IdealPrefixClosed still require direct arbitrary-prefix J.TraceCertCov coverage.

7.8 The J-to-K cutpoint bridge

Trace matching alone is insufficient for deadlock reasoning. Core.18 defines the cross-model K-facing correspondence as equality of one common typed Core.18c record <E,w,H,O>, not as a list of independently asserted source/RTL state clauses. The source record is KObs_E,w(sigma_ref); Appendix J constructs KObsPost_J, proves source realizability and complete request attribution through J.OfferReach/J.OfferConf, and J.KObsConf establishes typed equality for the exact selected package. Core.18d/e reconstruct the common K-facing facts. For the standard universal QSync route, J.QSyncCertState packages this full per-cutpoint payload (including J.RegPhaseReach and K.DrainReach) without assuming K.CertCov, and J.QSyncCertInd carries it from one canonical cycle cutpoint to the next.

Bridge element	Purpose
J.HCanon	Proves that the selected source and RTL prefixes have the same canonical replay history H modulo the permitted epsilon/history quotient.
J.RegPhaseReach	Records that the exact source witness was constructed with Core.RegSeq and the Core.Phase L1/L2/L3 discipline.
J.OfferReach	Shows that <E,w,H,O> is actually realized by a reachable source KCut selected by the exact NormRef witness, not merely a well-typed tuple.
J.OfferConf	At the selected RTL KCut, bidirectionally and uniquely matches every residual offer in O to a relevant asserted RTL request and every relevant RTL request back to exactly one offer. Core.SyncFence is load-bearing for identifying unqualified residual offers with the current-interval Pending_P set.

Bridge element	Purpose
J.KObsConf	Binds one exact source/RTL package with explicit NormRef/NormPost KCut endpoints, common I/E/w/H/O, complete request attribution, and boundary/reset/epsilon-opacity evidence. It is the downstream theorem obligation that establishes the Core.18 cross-model equality; the Core itself fixes only the common type/equality notion.
Corollary J.1	Defines the selected post record KObsPost_J(rho_post;E,w,H,O)=<E,w,H,O>, proves it equals KObs_E,w(sigma_ref), and through J.KObs-Proc/Chan/Offers/WFG exports the common reconstructed process/channel/frontier/request/enablement/WFG/snapshot-drain facts. It makes no K.Dead, waiver, terminal, or diagnostic conclusion; Appendix K factors those from the common record.

AUDIT CRITICAL: J.OfferConf request-to-offer completeness
Every relevant asserted RTL boundary request must be attributed to exactly one source residual offer at the selected explicit boundary. Omitting a hidden asserted request, duplicating attribution, or using an unrelated source cutpoint invalidates the KObs bridge. This is the implementation abstraction boundary on which Appendix K relies.

8. Appendix K - conditional deadlock preservation

PURPOSE	Preserve absence of reachable drain-closed closed internal message-passing wait cycles from the selected source-reference instance to covered RTL KCuts. Engineer takeaway: K is not a generic progress theorem. It proves a specific KCut property and is conditional on A5-L plus fairness, drain, environment, value, lowering, and coverage obligations. Universal RTL claims consume Core.KCutClosure_post and K.CertCov; in the standard QSync route Core.QSync-Post/Core.QSync-KCutCanonical derive the complete canonical KCut domain and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync derive package totality.

8.1 The deadlock predicate

Appendix K evaluates Core.KCut states, not arbitrary epsilon-quiescent states. A KCut is reachable, epsilon-quiescent for the relevant proof model, and settled at every explicit boundary used by BoundaryReq/ChannelEnabled/ChannelDisabled. Source L2 SettlePoints are KCuts by Core.SettleIsKCut. In the standard RTL K-facing QSync scope, QS2a requires every reachable post KCut to identify its unique same-cycle registered/input frame and designated reachable K-origin, with only qualified settling between origin and KCut; Core.QSync-KCutCanonical therefore proves every reachable KCut is exactly the unique Core.QSync-Post endpoint of that origin. Core.KCutClosure_post remains the downstream semantic interface. Appendix K's deadlock observation boundary includes every WFG-relevant explicit abstract message-passing boundary, while RTL micro-staging already absorbed into B(c) remains internal. Environment-facing channels and SyncChannel barriers are handled through separate scope/assumption rules. Definition K.Dead is the single authoritative ordinary deadlock predicate, with Dead_K^W the waiver-parameterized form. Appendix-K A9 and A10 are instance assumptions that refer back to the Core scalar endpoint-cardinality and reset-empty channel conventions; they do not restate competing definitions.

Object	Meaning

Object	Meaning
Front_P	The minimal pending blocking message-operation items of process P whose endpoint requests are currently asserted. Merely being the next source call is not enough; the request must have issued. Core.OrdFrontSingleton proves |Front_P| <= 1 in an ordinary non-elaborated source interval; multiple members require explicit Sys_B^elab partial-order structure.
Blocked process	Front_P is nonempty and every frontier item is ChannelDisabled at the selected explicit boundary. A disabled edge for one item does not by itself make a multi-frontier process blocked.
WFG edge P -> Q	For each disabled frontier item x of P, Core.WFG contains P -> Q where Q owns x's unique complementary endpoint. Multi-frontier processes may therefore have multiple outgoing edges, while the blocked-process condition remains an all-items-disabled test.
DrainClosedNow(D)	No already-offered non-frontier request owned by D remains enabled; otherwise finite downstream drain could still change the candidate wait structure.
Definition K.Dead / K.DeadW	K.Dead is the single authoritative ordinary reachable drain-closed closed internal wait-cycle predicate. Dead_K^W is the waiver-parameterized form excluding declared KObs-closed cutpoint classes. Optional Dead_KS remains diagnostic only; later assumptions do not redefine K.Dead.

SCOPE: Reachable closed internal wait-cycle cutpoint
The operative predicate is stronger than “the system is currently slow” and different from a global permanent-deadlock definition. It identifies a reachable internally closed wait cycle at a drain-closed snapshot. Escape through a nondefault timed/reactive environment co-frontier is a separate concern controlled by K.EnvMF and explicit waivers.

8.2 Reconstructing and factoring the wait state from J.KObs

Result	Function
J.KObs-Proc / Chan / Offers / WFG	Appendix J reconstructs process replay, explicit channel state, residual offers, frontier/request/enablement, Core.WFG, and snapshot drain facts from the common KObs record.
K.EnvTerminalAdequate / K.KObsDerived	Appendix K first supplies the environment-terminal adequacy premise and gathers the K-facing derived functions needed before deadlock factoring.
K.KObs-Dead / K.KObs-Ext	K.KObs-Dead factors K.Dead/K.DeadW and related K predicates from KObs-derived inputs; K.KObs-Ext transfers those K-owned predicates across equal KObs.
K.0 / K.1a	K.0 reconstructs the operational minimal pending blocking frontier from KObs-derived inputs. K.1a is the source-side/lowered-KCut channel-disabled characterization on the domain where K.Dead is defined; its buffered/rendezvous cases cite Core.12/Core.13 directly rather than asserting an RTL microstate theorem.
K.2 / K.2a	Use Core.WFG and the snapshot-drain functions to prove WFG and DrainClosedNow extensionality at equal KObs cutpoints.

8.3 One certified cutpoint versus universal coverage

DEFINITION: K.CertifiedCutpoint

A K-ready package for one reachable RTL KCut rho_post, together with at least one explicitly recorded reachable prefix tau_post and finite normalization witness nu_post satisfying NormPost(tau_post,nu_post,rho_post). It binds the theorem instance and contains the J.1/KObs correspondence, exact source KCut witness, J.RegPhaseReach, K.DrainReach including Core.SettleKBridge, point-to-point boundary facts, waiver closure, and the remaining K-facing assumptions for that one package. It does not imply K.CertCov.

THEOREM: K.1A - certified-cutpoint preservation from A5-L

Assume one certified RTL cutpoint satisfies Dead_K^W. Equal KObs plus the J-owned reconstruction and K.KObs-Dead/K.KObs-Ext transfer the same frontier/WFG/drain/deadlock predicate to the exact reachable Sys_ref witness. J.RegPhaseReach plus K.DrainReach prove that witness is Core.LAdm-admissible. This contradicts A5-L. Therefore that certified RTL prefix/normalization/cutpoint package cannot contain the non-waived internal wait cycle.

COROLLARY: K.1A-Total - universal result under K.CertCov and KCut closure

K.1A is pointwise over an already selected certified KCut. A universal claim additionally consumes Core.KCutClosure_post and K.CertCov. In the standard QSync scope, audited structural evidence including QS2a invokes Core.QSync-Post and Core.QSync-KCutCanonical, giving a canonical NormPost endpoint for every reachable KCut. The preferred package-totality route is not an unrelated per-cutpoint assumption: J.QSyncCertInd proves a base case plus one-cycle preservation over every reachable RTL successor; J.CertExist-QSync lifts that induction to J.QSyncCertCov and K.CertCov-QSync binds the tuples through K.CertHdr to derive K.CertCov. A single observed simulation path or sampled KCut set is insufficient unless a separately proved determinism/exhaustiveness result collapses the successor set. K.CertCov remains total only on KCutReach_post(I), not every arbitrary RTL prefix or normalization witness.

STANDARD QSYNC PACKAGE-TOTALITY ROUTE

J.QSyncCertState contains the substantive exact-prefix J/K package: canonical NormPost endpoint, accepted J.LocalGWR/J.GWR-Comp witness, J.HCanon/OfferReach/OfferConf/KObsConf, theorem-derived J.RegPhaseReach, and K.DrainReach/Core.SettleKBridge. J.QSyncCertInd is anti-circular: CI0 establishes the initial tuple; CI1 must extend it through any reachable one-cycle RTL successor; CI2 requires exhaustive successor coverage and may not assume the successor K.CertifiedCutpoint or K.CertCov. J.CertExist-QSync plus K.CertCov-QSync then supplies K.CertCov on demand over the canonical KCut domain. The same J.QSyncCertInd proof projects to the cycle-aligned J safety proof.

8.4 The central source-liveness premise A5-L

PREMISE: A5-L - scheduler-robust liberal source liveness

No finite Policy-L prefix admissible under Core.LAdm for the selected Sys_ref instance reaches a source K-facing cutpoint in KCutReach_ref(I) satisfying Dead_K^W. Drain-closedness is part of Dead_K itself, not an extra restriction on the A5-L cutpoint domain. Because Policy-S-shaped serial prefixes are included among admissible Policy-L prefixes, a design with a reachable serial wait cycle makes A5-L false; another proof technique cannot certify the unchanged instance.

A5-L is intentionally global. Internal wait cycles depend on interactions among processes, capacities, environment behavior, and the selected issue policy. Unlike the J safety construction, it cannot generally be reduced to independent per-process liveness theorems.

8.5 The primary Policy-S discharge route

The primary user-facing activity is still one selected complete instrumented Policy-S execution of the pre-HLS model, but the reduction exposes the formal interfaces that make that run probative. K.Low/K.Low.A5 handles the literal-blocking Sys_K form and transport back to Sys_ref. K.OpKey and K.CVPred/K.CV-WF supply common operation/fact keys, deterministic provenance, well-foundedness, and the comparison-availability closure needed by K.2b.P0/R/P; each recursively materialized non-transfer predecessor has an explicit Core.16 reach-legality disposition. K.EnvMF uses K.JointFreeze-Ord for ordinary/comparability-certified singleton frontiers and requires explicit K.JointFreeze only for admitted StandardEnv multi-frontiers. K.CompareAdvanceLegal covers both P0 preparation/launch and R Issue/assertion legality, with ordinary or comparability-certified singleton cases discharged via K.CompareAdvanceLegal-Ord. K.2b.E constructs the hypothetical fair Policy-L continuation using Core.CycleClosure, Core.FairPick/Core.FairCycleDovetail, B2/B3/B5 and K.Drain. Reset/termination are handled either candidate-by-candidate through the reset-first K.InterruptionGate schema or by one uniform U1-U3 instance-level Continue record. The flow separately establishes SDet, K.CompleteS, total K.RespCov, K.RespClean, and exact certificate binding.

Element	Role
K.Low / K.Low.A5 / Sys_K	Lower synchronization waits and other supported guard dependencies into literal blocking Sys_K structure, prove the local correspondence K.Low.C, and use K.Low.A5 to transport the scheduler-robust A5-L result from the lowered Sys_K instance back to the selected Sys_ref instance with explicit waiver pullback.
K.EnvMF	For StandardEnv/B1-available, ordinary or comparability-certified singleton frontiers use theorem-derived K.JointFreeze-Ord; every admitted Sys_B^elab multi-frontier candidate additionally supplies K.JointFreeze JF1/JF2 cross-process no-release evidence. Timed/reactive or otherwise nondefault environments are restricted to a certified single internal frontier for every relevant candidate.
K.JointFreeze / K.CompareAdvanceLegal	JointFreeze is an explicit certificate only for admitted StandardEnv multi-frontier components; ordinary/singleton frontiers cite K.JointFreeze-Ord. K.CompareAdvanceLegal covers the main serial-vs-liberal comparison advance for every K.2b.P0/K.2b.R-consumed owner/key, discharging both P0 ReachPrep/launch and R Issue/assertion Core.16 compatibility. Ordinary/singleton main cases cite K.CompareAdvanceLegal-Ord; explicit Sys_B^elab multi-frontiers need checked elaboration/order evidence. A prefix/key may not be simultaneously JointFreeze-suppressed and CompareAdvanceLegal-required.
SDet	A flow qualification that fixes every history-affecting choice - model, capacities, elaboration, reset, stimulus/environment, witness, arbitration, concrete simulator scheduling, Core.PolicyS issue semantics, and epsilon normalization - and identifies one run artifact rho_S^det. SDet alone does not prove completeness or response cleanliness.
K.InterruptionGate / K.DomResetScope / Reset / Terminal	Before the frozen-continuation argument, either certify candidate-relative K.DomResetScope/DomEpoch records (D3 covers every identity consumed by K.2b.P0/R/P) and evaluate Reset first then Terminal, or provide one fixed conservative U1-U3 instance-level scope proving all required Continue branches. DirectFail remains the independently mapped RW2 cancelled-pending or DeclaredTerminal maximal-pending response-failure path. If the applicable route cannot be proved, use another A5-L discharge.
K.RespCov	Default-total finite-bound coverage of every static internal process-facing blocking site and relevant dynamic class, with an unambiguous trigger/response definition and a validated monitor, static bound, or unreachability disposition.
K.RespClean	No finite prefix of the complete selected run witnesses a non-waived certified-bound expiration, maximal-finite pending frontier, cancellation/reset failure, or end-of-test pending failure.

Element	Role
K.CompleteS	The selected run is maximal and fair. A terminating instance reaches its declared terminal state with every monitor resolved; a nonterminating instance requires an invariant, sound model-checking/lasso proof, or equivalent completeness evidence.
A5-S / CF-S	The complete operational certificate over the SDet-selected run: K.CompleteS + total K.RespCov + K.RespClean, bound to one normalized theorem instance and trust boundary, together with the applicable lowering/provenance, Core.CycleClosure/Core.IssueFair, K.EnvMF, K.DomResetScope, and ordered Reset/Terminal gate evidence. This is the source-side package from which K.2c discharges A5-L on the supported fragment.
K.OpKey / K.CVPred / K.CV-WF / K.2b.P0-P1	OpKey supplies theorem-instance occurrence keys. K.CVPred supplies deterministic fact evaluators, finite predecessor sets, explicit occurrence closure, and the serial-route comparison-availability closure; transfer predecessors must already be matched, while lower-ranked non-transfer predecessors are recursively materialized under CVRank with an explicit Core.16 reach-legality disposition that preserves the frozen component/DomEpoch and introduces no unmatched dominance-domain endpoint transfer. K.CV-WF supplies the well-founded relation/rank. K.2b.P1 proves value/identity conditional on occurrence; K.2b.P0 proves keyed occurrence/reachability.
Core.CycleClosure / Core.FairPick / Core.FairCycleDovetail	K.2b.E constructs the frozen liberal continuation through typed Core.CycleResult successors. Core.CycleClosure C2/Core.CycleChoiceExt type the partial-cycle completions. Core.FairPick packages the existing Core.IssueFair and B2/B3 behavioral discharges as prefix-local selectors/starvation proofs; Core.FairCycleDovetail first completes the selected L1 segment through an actual L2 SettlePoint, then selects legal fair L3 work and completes the CycleResult. B5 remains a separate closure premise.

LEMMA: K.2b - deterministic serial-source dominance with response-failure extraction

Under the blocking/lowering, value-determinism/provenance, comparison-availability, environment/frontier-freeze, Core.16 comparison-advance, constructive fairness, drain, identity, reset/terminal applicability, and total-response-coverage premises, any Core.LAdm-admissible Policy-L source KCut satisfying Dead_K^W forces the same SDet-selected Policy-S execution to produce a finite non-waived K.RespFail witness. The strengthened proof carries both the earlier serial endpoint transfers and the lower-ranked K.CV predecessor facts needed by P0/R; predecessor materialization may not release the frozen component or synthesize missing prerequisite transfers. K.2b.E uses Core.FairPick/Core.FairCycleDovetail rather than assuming a pre-existing fair continuation. Failed non-blocking polls remain WC-selected return outcomes, not commits. The lemma does not claim equal traces, equal timing, or the same deadlock component.

COROLLARY: K.2c - the selected deterministic Policy-S execution suffices

If the complete selected Policy-S run has total response coverage and is response-clean, K.2b rules out any liberal Dead_K^W counterexample; therefore A5-S implies the lowered A5L_Sys_K result and K.Low.A5 transports it to the Sys_ref A5-L consumed by K.1A. The proof uses common K.OpKey keys; K.CVPred/K.CV-WF including comparison-availability and recursive Core.16 reach dispositions; K.EnvMF with JointFreeze where applicable; K.CompareAdvanceLegal; Core.CycleClosure/Core.FairPick/Core.FairCycleDovetail for the frozen continuation; and either candidate-specific reset-first gates or the uniform U1-U3 Continue discharge. SDet removes alternate Policy-S simulator histories.

STATUS: Paper proofs and the standard QSync package-totality theorem route supplied; Lean mechanization and concrete flow qualification outstanding

The instrumented Policy-S simulation and bounded-response-monitor workflow can be implemented now. Appendix K-P supplies the serial-reduction paper proofs, and the 16 August QSync additions supply Core.QSync-KCutCanonical, J.CertExist-QSync, and K.CertCov-QSync at paper level. Mechanization/checkers must still validate OpKey/CV equations and comparison-availability, recursive Core.16 reach dispositions, JointFreeze/CompareAdvanceLegal where explicit, FairPick/FairCycleDovetail, gate anti-circularity or uniform U1-U3 evidence, monitor coverage/waivers, lowering transport, and the proof-producing QSync/QSyncCertInd invariant for the concrete HLS flow.

PRACTICAL WARNING: One run must be complete and flow-qualified

For a terminating design, the selected run must reach the declared maximal terminal state and resolve every covered monitor. For a nonterminating design, a finite trace is only evidence; an invariant, sound model-checking result, recurrence/lasso proof, or equivalent argument must establish completeness and continued response cleanliness. In either case, the flow must also establish SDet, route applicability, total finite-bound response coverage, and certificate validity; a clean trace alone is not the theorem premise.

8.6 Alternate direct discharges of A5-L

Route	When it can discharge A5-L
K.Str-1 - acyclic dependency graph	If the full static process dependency graph is acyclic, no WFG cycle can occur.
K.Str-2 - ranked dependency	A well-founded rank strictly decreases along every realizable WFG edge, excluding cycles.
K.Str-3 - credit/token cycle breaking	Every static dependency cycle contains an edge proved unrealizable at reachable cutpoints, often by occupancy, initial-token, or credit invariants.
CF-3 / direct invariant	An inductive abstraction proves the prohibited Policy-L cutpoint unreachable.
CF-4 / exploration	Sound exhaustive or bounded model checking over the selected transition system.
CF-5 / tool certificate	A tool-emitted certificate using knowledge of the scheduled control graph, storage, arbitration, joins, and automatic flush.

These are alternate proofs of exactly the same A5-L property. They cannot rescue a design for which an admissible serial-shaped Policy-L prefix already reaches Dead_K^W.

Route-dependent continuation, reset, and termination coverage. The alternate A5-L routes prove the same abstract premise but need not use the Policy-S complete-cycle machinery. The Policy-S route requires K.CompleteS and its selected Core.CycleResult semantics, the constructive Core.FairPick continuation discharge, and reset/terminal interruption coverage. That coverage may be candidate-specific through K.DomResetScope plus reset-first K.ResetEpoch/K.TerminalDisposition records, or one uniform instance-level U1-U3 sufficient condition. A reset or terminal event that cannot be classified by the selected valid route puts that candidate outside the serial reduction; simulation stopping does not make it safe.

8.7 Standing assumptions A1-A11 in engineer terms

Assumption	Plain-language role
A1	The source and RTL satisfy the previously defined R-rules and B-rules at the selected boundaries.
A2	Finite internal epsilon depth / finite stutter when not externally stalled.

Assumption	Plain-language role
A3	Weak fairness: a continuously enabled action is eventually selected.
A4	Channel progress: persistent complementary requests eventually discharge full/empty/rendezvous blocking.
A5	For the user-facing Policy-S route, a complete bounded-response-clean selected serial run; K.1A itself consumes A5-L directly.
A6	Point-to-point channels with unique producer and consumer at every evaluated boundary.
A7	B1 RTL determinism/witness selection plus the exact equal-KObs source/RTL correspondence and Core.LAdm qualification.
A8	Barrier participation is well formed; barrier mismatches are outside the internal message-passing deadlock theorem.
A9	Instance assumption referring back to the Core scalar endpoint/cardinality convention: at most one Push and one Pop commit per channel per cycle unless an explicit multi-lane/burst model is selected. It does not redefine the Core rule.
A10	Instance assumption referring back to the Core reset-empty buffered-channel convention: every selected buffered channel is logically empty at each applicable reset boundary unless preload is explicitly modeled. It does not create a competing reset-state definition.
A11	K-epsilon: hidden epsilon steps do not create, remove, or redirect the WFG-relevant blocking structure.

THEOREM: K.1 - user-facing deterministic Policy-S preservation

Combine the uniform K.1A premises, SDet/A5-S and the K.2b/K.2c serial-route obligations: K.Low/K.Low.A5; common K.OpKey plus K.CVPred/K.CV-WF with comparison-availability; K.BF/Core.IC persistence; K-epsilon and K.Drain/Core.SettleKBridge; K.EnvMF with JointFreeze where needed; K.CompareAdvanceLegal; Core.CycleClosure/Core.FairPick/Core.FairCycleDovetail; the reset-first interruption gates or uniform U1-U3 discharge; K.RespCov and K.CompleteS/K.RespClean. Corollary K.2c supplies A5-L. For the universal RTL layer, Core.QSync-Post and Core.QSync-KCutCanonical derive the complete KCut domain and a checked J.QSyncCertInd feeds J.CertExist-QSync/K.CertCov-QSync to derive K.CertCov. K.1A-Total then excludes the prohibited persistent K-relevant closed internal wait-cycle configuration over the complete reachable RTL KCut domain. The conclusion remains narrower than 'RTL never hangs'.

Status. The paper theorem stack now includes the standard QSync canonical-domain and package-totality route. It is not yet a mechanized, flow-qualified signoff result: the concrete HLS flow still must provide/check QSync/QSyncCertInd, FairPick, provenance/comparison, frontier, simulator/lowering, response, and certificate evidence. K.1A remains the direct interface when A5-L is discharged by another valid route.

8.8 What Appendix K does not cover

- Barrier mismatches or conditional SyncChannel participation (handled by A8 and separate protocol verification).
- Waiting indefinitely for an external environment input, output readiness, starvation/perpetual under-supply, termination, EOF, cancellation, or reset behavior not represented by the defined closed internal WFG/Dead_K predicate. The persistent-deadlock conclusion is deliberately limited to that internal wait-cycle class.
- Independent nonterminal withdrawal, cancellation, retry, or reattribution of an already-issued blocking request; Definition Core.IC excludes those lifecycles from Sys_ref unless a separate operational/correspondence/progress extension is defined, and the RTL side must separately demonstrate Core.IC conformance.
- Cycle-accurate RTL response bounds; K.Resp evidence is a source-side discharge of A5-L, not an RTL timing contract.
- Designs whose correctness requires favorable eager/same-cycle issue when a serial-shaped admissible prefix deadlocks.

9. Appendix L - witness selection and snooping

PURPOSE	**Conditionally realize a WC-compatible source witness for latency-sensitive epsilon choices while leaving the unique free-running RTL behavior untouched.** **Engineer takeaway:** Snooping selects one admissible source witness only when the required WC/L.Dir witness exists. It must not feed pre-HLS readiness back into RTL, relabel payloads, drop/reorder observations, or treat an unrealizable witness as a silent stall.

9.1 Why snooping is needed

A non-blocking arbiter, interrupt controller, or retry/timeout block may make a later observable choice based on which request arrived first or how many polls failed. HLS changes internal latency, so the raw pre-HLS and post-HLS simulations can select different epsilon outcomes even though both satisfy the broad scheduling contract. When those outcomes affect later Sigma-visible behavior, Appendix I needs a specific WC-compatible witness.

If NB-CF holds - failed non-blocking outcomes do not affect later observable control, values, frontier, or request attribution - the witness is trivial and snooping is unnecessary. Cadence-sensitive polling is different: it is a localized latency-sensitive protocol and must be isolated or explicitly snoop-aligned. Appendix L is conditional: if no admissible source execution can consume the RTL event sequence in ordinal order with matching kind and payload, WC/L.Dir is not discharged for that comparison.

DEFINITION: L.1 - witness selector / snooping

At each relevant epsilon-quiescent source witness-choice cutpoint, the selector returns the outcome chosen by the matched RTL prefix using only causally available history. This is intentionally a generic quiescent witness-selection interface, not a Core.KCut requirement: KCut adds a K-facing boundary-evaluation condition that an arbitrary snooped epsilon choice need not satisfy. The selector cannot depend on future RTL events or invent a retry/timeout outcome inconsistent with the isolated protocol.

LEMMA: L.2 - conditional snooping discharge of WC

If the snooped set covers every epsilon choice that can affect later observable behavior, the wrapper changes only epsilon-choice selection, selected outcomes are causal and RTL-consistent, successful and failed non-blocking calls retain the required Sigma/epsilon interpretation, cadence-sensitive blocks are isolated, and the selected ordinal event stream is realizable, then the selected source execution satisfies WC for Appendices I and J. This is a conditional witness-construction theorem, not a guarantee that every RTL event stream has a matching source witness at the chosen boundary.

9.2 One-directional architecture

CONDITION: L.Dir - one-directional alignment / RTL-side non-interference

The RTL_B execution is free-running under the fixed stimulus. The pre-HLS snoop sequence must consume the recorded RTL snoop sequence in ordinal order, with matching kind and payload, and no pre-HLS backpressure/readiness may stall or alter RTL. Benign cycle skew is permitted. Order, kind, or payload divergence means that the required WC witness does not exist at the chosen boundary; the run has no theorem-backed guarantee and the harness must abort and triage the failure rather than silently gating or hanging.

9.3 Wrapper realization requirements W1-W5

Requirement	Engineering meaning
W1 - recorded ordinal observation	Capture each RTL snooped event by dynamic ordinal and hold it until the pre-HLS side consumes the same ordinal. Do not rely on overlapping live assertion windows.
W2 - gate only commit enable	Gate only the mirror signal that enables the pre-HLS commit: observed valid for a pre-HLS Pop or observed ready for a pre-HLS Push. Never modify the source endpoint-owned request.
W3 - registered sampling	Register the RTL observation before it enters the source-side gate to avoid delta-cycle races; the extra source latency is part of witness selection.
W4 - compare and watchdog	At each source snoop commit, compare ordinal, kind, and payload against the recorded RTL event. Never substitute the RTL payload into the source, because that would hide value divergence. Detect missing, extra, reordered, or never-consumed observations. A watchdog must convert non-existence or failure to realize the witness into a reported result; if the watchdog bound is not proved sufficient, a bound-exceeded abort is an inconclusive witness-realization failure, not by itself a proved L.Dir violation.
W5 - independent coverage proof	The wrapper mechanism is sound only for the points it records. A separate analysis must prove that all epsilon choices capable of affecting later observables are covered.

IMPLEMENTATION PITFALL: Why a live AND is unsound

ANDing the current pre-HLS and current RTL handshake levels can drop an RTL event when the RTL assertion retires before the source arrives. It can also reorder or duplicate matching under multiple outstanding events. The normative rule is an AND against a recorded per-ordinal observation retained until source consumption.

9.4 Harness soundness and shared testbenches

LEMMA: L.5 - online realization of the witness selector

A co-simulation harness realizes the abstract selector only when a matching witness exists, the harness preserves the free-running RTL trace, implements W1-W5, satisfies L.Dir, and produces the same source-side witness choices assumed by the formal comparison. The formal theorem is about the fixed RTL trace and selected source witness; harness correctness, coverage, and successful witness realization are additional implementation obligations.

A shared reactive testbench becomes problematic once one model can lag significantly, because reactions may be driven by whichever model the testbench observes. The flow should replicate the stimulus/testbench per model, or structure a shared environment around a common causal transaction stream rather than raw cycle timing.

9.5 What snooping means formally

Snooping does not use RTL to change the source specification. The source model is intentionally under-specified with respect to permitted latency and epsilon choice. When a WC/L.Dir witness exists, the RTL observation selects one admissible execution of that specification for comparison. If the event order, kind, or payload cannot be realized, the comparison is outside theorem scope and must be triaged among source-model ill-formedness, incomplete snooping coverage/harness behavior, and tool/RTL non-compliance.

9.6 Latency-robustness stress testing

Appendix L also recommends randomized stalls, varied channel latency/capacity, and adversarial weakly fair scheduling in the pre-HLS model. This is not a substitute for the formal premises, but it is a practical way to detect designs that accidentally rely on incidental ordering of causally independent events. A failure under such stress usually indicates that the source specification itself is outside the intended latency-insensitive model or that an explicit snoop/isolation boundary is missing.

10. How the appendices work together in a verification flow

PURPOSE	**Translate the theorem architecture into a concrete signoff workflow.** **Engineer takeaway:** Safety-only use stops after the J.1/J.1-Safe package. Appendix K is added only when the claim includes the defined internal wait-cycle freedom, and it requires substantially more global evidence.

10.1 Step-by-step safety flow

Step	Deliverable
1. Fix the theorem instance	First screen the design patterns against Appendix F of the scheduling-rules document. Then select one single-global-clock theorem instance: Sys_ref = Sys_B or Sys_B^elab, explicit boundaries, capacities, Core.ElabProj package where needed, observable alphabet, reset interpretation, fixed stimulus/environment, witness provenance, and tool/RTL identities. Multi-clock designs require a separate semantic layer or per-domain verified boundaries.
2. Establish Core/G and QSync conformance	Check the Core/G instance semantics and qualifications - Core.8 E4 commit mapping, signal anchors, Core.RegSeq, Core.ReachPrep where eager preparation is admitted, the RTL conformance counterpart of Core.IC continued-request lifecycle/attribution, Core.SyncFence, Core.MemSyncOrder where array/memory accesses cross synchronization, Core.LLegal/Core.Phase/Core.Settle and the L2 frame, Core.17 ΣMatch including selected proof-internal RESET units and R2C component/local-occurrence matching, R1-R5/E1-E5, channel/elaboration boundaries, and Core.QSync QS1-QS5 including QS3 realization/adequacy. On Sys_ref, Core.IC/Core.SyncFence are transition legality; on RTL they are evidence-backed conformance. Bind the single global clock and fixed cycle-entry/environment choices.
3. Discharge WC	Use NB-CF, an Appendix L snooping/L.2 construction with a realized WC/L.Dir witness, direct witness construction, or a complete mixed per-site coverage argument.
4. Produce local evidence	Generate J.PCert from Appendix I and per-channel/atomic/environment/reset certificates plus typed footprints and local dependencies.
5. Run J.GWR-Comp	On the normal route, invoke Core.QSync-Ref for source L2 settling, generate the D1-D4 semantic edges and D5 Horizon-stratification edges, validate D5-only commutation with J.HorizonOrderSafe, prove the graph well-founded with J.FoundationRank/J.DepRank, then build the phase-normal Horizon-batched finite-ideal chain and J.GWR/J.RegPhaseReach. No generic infinite fair extension is part of this finite-prefix construction.
6. Apply Proposition J.0 and Theorem J.1	Use Proposition J.0 on the compatible source/RTL pair constructed by J.GWR-Comp (or supplied by the direct J.GWR route), then apply Theorem J.1 to obtain Post -> Ref observable compatibility for the covered reachable RTL prefix over Reach_post/Reach_ref. Core.17 ΣMatch supplies complete selected occurrence/value matching; Appendix G supplies E1-E5. In the restricted Ref -> Post clause, J.RTLConsistentRefPrefix instead supplies the already-selected reachable matching RTL prefix with no unmatched selected observable unit and the required annotations, and J.0 composes that pair.

Step	Deliverable
7. Transfer the safety property	Use route-sensitive J.TraceCertCov and Corollary J.1-Safe over the required RTL coverage domain, with the safety predicate satisfying J.IdealPrefixClosed. For the standard QSync route, discharge J.QSyncTraceInd (or project an already-proved J.QSyncCertInd through J.QSyncCertInd-Trace), then use J.TraceCertCov-QSyncCycle/J.1-Safe-QSyncCycle on CycleReach_post(I). Core.RTLCycleCarrier plus J.CycleIdealCover/J.1-Safe-QSync then extend that conclusion to all reachable finite outer observable prefixes without constructing a separate package for every intra-cycle ideal. This shortcut does not cover a property that fails J.IdealPrefixClosed, such as some cross-interface scoreboards; those claims require direct J.TraceCertCov on the relevant arbitrary-prefix domain. If one complete source run represents source verification, also prove J.RefProjCover.

10.2 Additional liveness flow

Step	Deliverable
8. Establish the KCut observation domain	For source models in the standard scope, Core.QSync-Ref proves the finite unique L2 SettlePoint and Core.SettleIsKCut makes it the source K-facing state. For RTL_B, establish QSync including QS2a origin recoverability and invoke Core.QSync-Post/Core.QSync-KCutCanonical: every reachable post KCut is the unique same-cycle endpoint of its designated reachable K-origin and has a canonical NormPost witness before the next registered update. Core.KCutObserve preserves any persistent K-relevant blocker under K-epsilon.
9. Build the KObs bridge	Discharge J.HCanon, J.OfferReach, J.OfferConf, and J.KObsConf for the exact same source/RTL/witness package and NormRef/NormPost KCut endpoints. Core.18 supplies the common Core.18c record type/equality; J supplies KObsPost_J and proves KObsPost_J = KObs_E,w(sigma_ref).
10. Qualify the source witness	Use theorem-derived J.RegPhaseReach plus application evidence K.DrainReach to prove K.RLAdmReach/Core.LAdm admissibility for the exact source witness. K.DrainReach must include Core.SettleKBridge from every persisting L2 activation through intervening permitted drain to the selected later KCut.
11. Certify the cutpoint	Assemble K.CertifiedCutpoint for the selected RTL KCut and one recorded reachable-prefix/NormPost pair, including point-to-point boundaries, waiver closure, exact source KCut witness, J.RegPhaseReach, K.DrainReach/Core.SettleKBridge, QSync theorem/evidence provenance where used, and all remaining K-facing assumptions.
12. Run and qualify the A5-L discharge	For the primary route, execute one complete instrumented Core.PolicyS run. Separately establish SDet; K.Low/K.Low.A5; K.OpKey; K.CVPred/K.CV-WF including comparison-availability and recursive Core.16 reach dispositions; K.EnvMF plus explicit K.JointFreeze only for admitted StandardEnv multi-frontiers; K.CompareAdvanceLegal (ordinary or comparability-certified singleton cases use K.CompareAdvanceLegal-Ord); Core.CycleClosure/Core.FairPick/Core.FairCycleDovetail/B5; and either candidate-specific K.DomResetScope + reset-first gates or the uniform U1-U3 gate record. Add total K.RespCov, K.CompleteS, K.RespClean, and exact certificate binding. K.2b/K.2c then discharge A5-L on the supported fragment.

Step	Deliverable
13. Establish universal KCut closure and coverage	Apply K.1A directly to one selected certified KCut. For K.1A-Total or universal K.1, consume Core.KCutClosure_post and K.CertCov. In the preferred standard QSync route, QSync-Post/QSync-KCutCanonical derive the complete canonical KCut domain, while J.QSyncCertInd proves the full per-cutpoint correspondence package by base/one-cycle/exhaustive-successor induction; J.CertExist-QSync then gives J.QSyncCertCov and K.CertCov-QSync derives K.CertCov. Alternate independently checked K.CertCov routes remain permitted. A single observed run or sampled KCut set is not package-totality evidence.

10.3 Recommended reading order in the scheduling-rules document

- Start with the front Abstract and Formal Guarantees at a Glance, then read Appendix F for the architect-facing design constraints. Use Appendix M for the formal conclusions, scope, and division of responsibilities. Continue with the unlettered Guide to the Formal Appendices, the Normative Formal Core for the shared semantic skeleton, and Appendix G for the R/E observable contract.
- In the Normative Formal Core, focus on Sys_ref, Core.8 Commit/CommitCycle, Core.ReachPrep, Core.IC, Core.RegSeq, Core.PolicyL/Core.LLegal/Core.Phase/Core.LAdm, Core.PolicyS, Core.IssueFair, Core.Settle and its L2 frame, Core.QSync/Core.QSync-Ref/Core.QSync-Post including QS3 and RTL QS2a, Core.QSync-KCutCanonical, Core.CycleState/Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure/Core.CycleChoiceExt, Core.FairPick/Core.FairCycleDovetail, Core.SyncFence, Core.MemSyncOrder, Core.ElabProj/Core.WFG/Core.16/Core.Drain/Core.SettleKBridge, Core.17 ΣMatch, and Core.18/Core.18c-e KObs. Read the Core.IC/Core.SyncFence status note as a trust-boundary rule: source legality is definitional; RTL conformance is evidence-backed.
- Read Appendix I to understand exactly what is local and why deferred non-blocking holes are needed.
- Read Appendix J as the central safety construction; focus first on J.LocalGWR, the D1-D4/D5 distinction in J.DepJ, J.DepSound/J.HorizonOrderSafe/J.DepComplete/J.DepRank/J.DepAcyclic, finite canonical ideals, J.RegPhaseNF, J.UnitExt, J.GWR-Comp, Theorem J.1 and J.RTLConsistentRefPrefix, J.IdealPrefixClosed, J.TraceCertCov/J.RefProjCover/J.1-Safe, then the QSync safety layer J.CycleReach_post/J.QSyncTraceInd/J.TraceCertCov-QSyncCycle/J.1-Safe-QSyncCycle/Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync and the richer J.QSyncCertState/J.QSyncCertInd/J.CertExist-QSync package induction, and finally J.HCanon/J.OfferReach/J.OfferConf/J.KObsConf plus J.KObs-Proc/Chan/Offers/WFG.
- Read Appendix L before K when witness-selected non-blocking or latency-sensitive interfaces are present.
- Read Appendix K for Core.WFG-based deadlock reconstruction, K.EnvTerminalAdequate/K.KObsDerived, K.KObs-Dead/K.KObs-Ext, the certified-cutpoint and K.CertCov interfaces including K.CertCov-QSync, A5-L, K.Low/K.Low.A5, K.OpKey, K.CVPred/K.CV-WF comparison-availability, K.2b.P0/P1, K.JointFreeze-Ord/K.JointFreeze, K.CompareAdvanceLegal/K.CompareAdvanceLegal-Ord, K.DomResetScope, K.InterruptionGate with candidate-specific or uniform U1-U3 reset-first discharge, and the Core.FairPick complete-cycle Policy-S response workflow.

11. Assumptions and evidence matrix

PURPOSE	**Summarize the named assumptions that most often determine whether a theorem instance is usable.** **Engineer takeaway:** The important audit question is not merely “is the assumption plausible?” but “what artifact establishes it for this exact source, RTL, capacity/elaboration, stimulus, witness, reset, and environment instance?”

Architect-facing entry point. Appendix F of the scheduling-rules document is the design-pattern entry point to this matrix. Poll-dependent control triggers WC/L.Dir or isolation evidence; cross-process memory triggers J.Mem, while array/sync ordering independently triggers Core.MemSyncOrder; eager source/RTL overlap triggers Core.ReachPrep coverage; ordinary blocking interfaces trigger the Core.IC implementation-conformance check even though Core.IC is constitutive on Sys_ref; standard K-facing QSync additionally triggers QS2a origin recoverability; and the Policy-S route adds K.CV comparison-availability, the applicable JointFreeze/CompareAdvanceLegal checks, constructive Core.FairPick continuation evidence, and reset/terminal interruption coverage. Universal standard-QSync K claims additionally use J.QSyncCertInd. Appendix M.7a remains the normative per-instance discharge ledger; M.7b is the standard four-bundle packaging view over that ledger.

How to read this matrix. The entries below are not a checklist of new manual tasks for every architect or design. The underlying engineering conditions represented by many entries were largely already present, explicitly or implicitly, in both HLS-generated and manual RTL flows; the formalism exposes them so a flow can determine applicability, assign ownership, and retain auditable evidence. Depending on the selected claim, an entry may be inapplicable, satisfied by a simple design restriction, derived from another checked result, discharged once through reusable tool, library, or template qualification, or established automatically for the particular implementation. The theorem-backed assurance layer can add explicit certificate, provenance, or checking work, but that additional assurance should not be confused with creating the underlying engineering condition.

Premise / evidence	Where it matters and typical discharge
B1 - RTL deterministic trace	Consumed by J/K/L selected-trace reasoning. Evidence: RTL/tool determinism proof, qualification, deterministic arbitration/environment setup, or a validated witness-selection setup. B1 is observable trace determinism, not the same thing as QSync same-cycle settling determinism.
B2 - weak fairness	Consumed by I progress embedding and K drain/progress arguments. Evidence: scheduler/arbiter fairness proof, assertions, or Appendix N pattern qualification.
B3 - channel progress	Consumed by I.2 and K. For Push at a full buffered channel, B3 guarantees a complementary Pop discharge that frees space; for Pop at empty, it guarantees a complementary Push discharge that supplies data. This is not by itself a conclusion that the original blocked operation commits. If that request remains pending and continuously enabled afterward, E4 plus B2 govern its eventual commit. Evidence may be a per-boundary progress certificate or the stated persistent two-endpoint-request premises.
B4 - finite epsilon-step depth	Consumed by Appendix I finite preparation and replay construction. D_P bounds consecutive P-owned epsilon-steps only while P remains internally advancing toward its next observable action and is not externally stalled. Stable waiting, peer delay, environmental delay, and back-pressure waiting are outside this charge. Evidence: finite source-control/pipeline/FSM progress plus the required no-infinite-epsilon-spin argument. B4 is not the QSync same-cycle settle theorem.
B5 / B6-Terminal	B5 supplies bounded global stabilization where invoked. A B6 idle suffix is permitted only after Core.TerminalIdle proves both a genuine B5/global fixed point and permanent no-future-enabling machine/environment behavior (Core.EnvTerminal supplies only the environment component). A currently quiet nonterminal cycle is Core.SilentCycle, not B6 stutter.
WC	Consumed by I and J; provenance may be NB-CF, L.2 snooping, direct witness construction, or mixed per-site coverage.
Core.RegSeq	Consumed by I/J/K. Evidence: sequential interface structure; request/data outputs are functions of registered state and operands fixed before L2; ordinary combinational derivation from those registers is allowed, but no same-cycle newly returned interface result may feed a dependent sequential output.

Premise / evidence	Where it matters and typical discharge
Core.QSync / QSync-Ref / QSync-Post	Normal J.LocalGWR -> J.GWR-Comp uses source QSync-Ref to obtain finite L2 settling. Standard QSync safety uses J.QSyncTraceInd over RTL K-origins, then Core.RTLCycleCarrier/J.CycleIdealCover to extend J.IdealPrefixClosed safety from cycle-aligned histories to all finite outer observable prefixes. Corollary J.1 cutpoint transfer and Appendix K use post-side QSync; the standard K-facing RTL audit includes QS2a origin recoverability in addition to QS1-QS5/QS3 realization. Core.QSync-KCutCanonical then identifies the complete reachable post-KCut domain. Whole-source Sys_B^elab qualification also needs checked template/design settling composition.
J.QSyncTraceInd / J.QSyncCertInd	QSyncTraceInd is the safety-only base/one-cycle/exhaustive-successor certificate over CycleReach_post and carries only the exact LocalGWR/J.GWR-Comp + J.HCanon state needed by safety. QSyncCertInd is the richer K-facing induction carrying OfferReach/OfferConf/KObsConf, RegPhaseReach and K.DrainReach as well. J.QSyncCertInd-Trace projects the latter to the former. For either induction, one observed simulation path is insufficient unless independent determinism/exhaustiveness collapses the reachable successor set.
J.LocalGWR	Consumed by the normal J.GWR-Comp route. Evidence: complete local process/channel/atomic/env/reset records, footprints, dependencies, agreement, and coverage.
B-faithfulness / Core.ElabProj	Consumed by J/K at every explicit abstract boundary. Evidence: back-annotation, structural RTL checks, interface monitors, refinement checks, or tool certificates. Elaborated instances additionally need Core.ElabProj evidence for trace projection, occupancy projection, conservation/accounting, and proof-internal WFG visibility.
J.TraceCertCov	Consumed by J.1-Safe over a selected RTL-prefix domain D. Evidence is route-sensitive. On the normal per-prefix route, each prefix carries accepted J.LocalGWR/J.GWR-Comp plus source QSync-Ref and remaining Post -> Ref premises. For the standard cycle-aligned QSync domain CycleReach_post, J.QSyncTraceInd and J.TraceCertCov-QSyncCycle derive the package coverage; Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync then extend J.IdealPrefixClosed safety to all reachable finite outer observable prefixes without adding per-prefix packages. A property that fails J.IdealPrefixClosed still requires direct J.TraceCertCov on its needed arbitrary-prefix domain. Safety coverage does not imply K.CertCov.
Core.ReachPrep / Core.16 / K.Drain / Core.SettleKBridge / K-epsilon	ReachPrep is required whenever Sys_ref permits source-later dependency-independent preparation before an earlier blocking call commits, and every RTL prefix in Post -> Ref coverage must be realizable by the selected ReachPrep semantics used by the accepted J construction. Core.16's no-new-later-work 'launch' prohibition is broader than message Issue. K.Drain/DrainReach must prove finite non-replenishing cleanup (finiteness is not inferred from capacity alone), the Settle-to-later-drain-closed-KCut bridge, request persistence, and no hidden WFG-changing epsilon behavior.

Premise / evidence	Where it matters and typical discharge
J.OfferReach / J.OfferConf / J.KObsConf	Consumed by Corollary J.1 and all K cutpoint reasoning. Evidence must bind the exact same source/RTL prefix, witness, E/w/H/O, and NormRef/NormPost endpoints. Core.18 fixes the common typed record/equality interface; J.KObsConf is the theorem-level proof that J's KObsPost_J representation equals the realized source KObs.
K.CertCov	Consumed by K.1A-Total and universal K.1 together with Core.KCutClosure_post. In the preferred standard QSync route, QS2a + Core.QSync-KCutCanonical identify the complete KCut domain, J.QSyncCertInd proves the full per-cutpoint package by base/one-cycle/exhaustive-successor induction, J.CertExist-QSync gives J.QSyncCertCov, and K.CertCov-QSync derives K.CertCov. Alternate total-extraction/refinement/exhaustive routes remain valid. One CF-0 package, sampled KCuts, or a clean simulation is insufficient.
A5-L	Consumed directly by K.1A. Evidence: Policy-S route through K.2c or a valid structural/direct/invariant/exploration/tool proof.
Selected Policy-S run / SDet / A5-S	Consumed by K.2b/K.2c/K.1. The user artifact is one complete instrumented serial-issue run with total finite-bound response monitoring. The flow separately supplies SDet; K.Low/K.Low.A5; K.OpKey/K.CVPred/K.CV-WF including comparison-availability/recursive Core.16 dispositions; K.EnvMF with explicit JointFreeze only for admitted StandardEnv multi-frontiers; K.CompareAdvanceLegal (ordinary or comparability-certified singleton cases use K.CompareAdvanceLegal-Ord); Core.CycleClosure/Core.FairPick/B5; candidate-specific reset-first gates or uniform U1-U3 evidence; completeness, coverage/bounds, response cleanliness, and exact certificate binding. Universal RTL coverage is handled separately by the QSync/QSyncCertInd package-totality layer.
K.EnvMF	Consumed by the serial route. StandardEnv/B1-available is allowed with ordinary/singleton frontiers through K.JointFreeze-Ord and with admitted explicit multi-frontiers only when K.JointFreeze JF1/JF2 is certified. Timed/reactive/nondefault environments require a certified single internal frontier for every relevant candidate; a nondefault multi-frontier candidate is outside the Policy-S reduction.
K.OpKey / K.CVPred / K.CV-WF / additional K.CV observation checks / K.Low	The common OpKey/CVFact provenance obligation is unconditional whenever the Policy-S serial route is used. Supply partial per-execution realization maps, key-level source order, finite Pred_CV sets, occurrence closure, deterministic Eval equations including none/some occurrence selection, and K.CV-WF/rank. Also discharge the comparison-availability closure: after required earlier transfers are matched, every predecessor fact needed by P0/R/P must be obtainable with the same keyed value without releasing the frozen component, changing DomEpoch, or introducing unmatched dominance-domain endpoint transfers; each non-transfer predecessor reach carries an explicit Core.16 disposition. K.Low remains a separate lowering/transport condition.

Premise / evidence	Where it matters and typical discharge
K.DomResetScope / K.InterruptionGate / K.ResetEpoch / K.TerminalDisposition	Consumed by the Policy-S serial reduction. Candidate-specific records must validate D1-D4; D3 covers every process-owned endpoint operation and explicit-boundary transfer identity consumed by K.2b.P0/R/P. Evaluate Reset first, then Terminal only after Reset Continue. Alternatively, one conservative uniform U1-U3 instance-level scope may prove all required Continue branches at once. DirectFail remains the independently mapped RW2 cancellation or DeclaredTerminal maximal-pending failure path.
L.Dir and W1-W5	Consumed by the online snooping harness. Evidence: free-running RTL, ordinal recording/consumption, comparison/watchdog, noninterference, and complete choice-point coverage.
ClockScope - single global clock	Applies to the G-K theorem stack and directly discharges QS1 evidence. The selected theorem instance uses one global synchronous cycle relation; CDCs are excluded from that instance or represented through separately specified/verified crossing components. Appendix O is only an informative multi-clock sketch until a separate semantic layer is supplied.
Core.SyncFence / R3/E5 completion fence	Core.SyncFence is constitutive synchronization-completion legality on Sys_ref and a separate RTL conformance obligation. Every blocking q in pref_S(s) has a designated process-facing commit no later than sync s. Same-cycle q/s completion is allowed; on RTL, ordering "before interval advance" is annotation/certificate accounting rather than physical sub-cycle timing. Typical evidence: R3/E5 trace checks, control-state assertions, boundary mapping, or formal refinement.
Core.MemSyncOrder	Separate local scheduling/mapping qualification whenever an array/memory access is source-ordered with a selected synchronization call, whether storage is internal or external. Verify mapped storage commits before s no later than s and after s strictly later, including any merge/split/reorder transformation evidence. It does not add memory events to Σ or become an Appendix I/J.1 replay premise; J.Mem is separately required for cross-process shared-memory value flow.
Core.KCutClosure_post / Core.KCutObserve	Consumed by K.1A-Total and universal K.1, separate from K.CertCov. In the standard user-facing scope, Core.KCutClosure_post is a theorem-derived interface: accepted RTL-side Core.QSync evidence invokes Core.QSync-Post, giving a finite same-cycle normalization to a unique PostKCut before the next registered update. K-epsilon plus Core.KCutObserve preserves any persistent relevant request/frontier/enablement configuration into that KCut. The former arbitrary invariant/checked-normalization routes are not independent standard-scope discharges; a non-QSync model needs a separately qualified extension.
Core.IC - request lifecycle / Issue-Commit conformance	Core.IC is constitutive Sys_ref transition legality, not an unconstrained proof axiom. On RTL_B, establish that every admitted blocking Push/Pop request is correctly attributed, remains asserted with stable Push payload after Issue until process-facing Commit or an affecting RW2 RESET, obeys same-endpoint/back-to-back attribution, and is not silently cancelled, withdrawn, retried, or reattributed. Evidence may be interface assertions/monitors, schedule/control-state evidence, request-attribution certificates, or formal refinement. K.BF consumes this persistence but does not own the conformance obligation.
K.BF - blocking-only serial fragment	Consumed by K.2b/K.2c/K.PSF/CF-S on the bounded-response serial route. In addition to citing established Core.IC/RW2 persistence, show every WFG-relevant wait is an

Premise / evidence	Where it matters and typical discharge
	ordinary blocking Push/Pop on a finite B-faithful point-to-point boundary, local computation cannot diverge before the process-facing frontier, and shared arbitration is explicit and deterministic or witness-fixed. This is a K-route applicability condition, not the definition of request persistence itself.
Core.CycleResult / Core.CycleClosure / Core.CycleChoiceExt / Core.FairPick	Required when K.2b.E constructs a complete/fair source continuation, not by finite-prefix J.1. Bind the full cycle-entry/next-state semantics and C2 partial-cycle extension. Core.FairPick packages the existing Core.IssueFair/B2/B3 discharges as prefix-local issue/boundary selectors with starvation-freedom and fixed-scheduler/environment legality; Core.FairCycleDovetail uses two CycleChoiceExt completions around the actual L2 SettlePoint to produce each typed CycleResult. B5 remains separate.

11.1 A practical certificate boundary

A useful implementation is a versioned package with a common header binding every artifact to one theorem instance: source and RTL build IDs, selected Sys_ref/Sys_K, explicit boundaries/capacities/elaboration, observation projection, stimulus/environment/reset policy, witness, Core.MemSyncOrder where applicable, Core.IC/Core.SyncFence conformance references, QSync evidence provenance, and the exact Core.18 KObs package. For standard universal QSync coverage it should also bind the QSyncCertInd base/step generator or proof and exhaustive-successor justification. For Policy-S it should additionally bind the OpKey/CVFact universes, K.CV-WF plus comparison-availability/recursive Core.16 dispositions, Core.CycleResult/Core.CycleClosure and FairPick evidence, K.EnvMF/JointFreeze/CompareAdvanceLegal references, either candidate-specific DomResetScope/gates or one uniform U1-U3 record, response-monitor configuration, run completeness, and waivers. The checker should reconstruct derived state and validate subordinate evidence rather than accepting a final Boolean claim. Appendix M.7b standardizes the logical grouping of this evidence without prescribing a physical file format.

SIGNOFF RULE: Do not mix witnesses or cutpoints
A J.OfferReach proof for one source state, a J.OfferConf proof for another RTL cycle, and a K.Drain proof for a different witness do not compose. The formal architecture repeatedly requires exact identity of the theorem instance, stimulus, elaboration, witness, H, O, source prefix, RTL prefix, and normalized cutpoints.

11.2 Standard QSync certificate profile

Appendix M.7b provides a normative packaging profile over the detailed M.7a discharge ledger for the standard normal-compositional QSync flow. The four names below are logical certificate interfaces only: they do not introduce new theorem predicates, merge or weaken obligations, make required conditions optional, or prescribe one physical file format. M.7a remains definitive. A bundle may reference theorem-derived facts instead of regenerating them, and every artifact used in one claim must map unambiguously to its M.7a obligations and share the same K.CertHdr/K.CertPkg identity/trust binding.

Logical bundle	What it owns in the architecture
1. Instance and QSync qualification	The theorem-instance identity and structural/scheduling/mapping qualifications: ClockScope; source/RTL QSync QS1-QS5 (including QS2a when K-facing); Core.RegSeq/ReachPrep; triggered Core.IC/Core.SyncFence RTL conformance; R1-R5/E1-E5; selected boundaries, capacities, B-faithfulness/Core.ElabProj; and applicable memory, reset, direct-input, and elaboration/template-composition qualification. It may cite QSync-Ref/QSync-Post/QSync-KCutCanonical after their inputs are checked. It does not itself establish J correspondence, KObs equality, K.CertCov, or A5-L.

Logical bundle	What it owns in the architecture
2. J correspondence/refinement	J.LocalGWR/J.GWR-Comp or a permitted J.GWR-Direct package, WC/witness binding, and J.HCanon. Safety form: J.QSyncTraceInd -> J.TraceCertCov-QSyncCycle/J.1-Safe-QSyncCycle, then Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync for all finite J.IdealPrefixClosed safety. K-facing form: J.QSyncCertInd/QSyncCertState with OfferReach/OfferConf/KObsConf, theorem-derived RegPhaseReach provenance, and the exact K.DrainReach/Core.SettleKBridge field. QSyncCertInd-Trace means the K-facing bundle can supply the safety induction rather than duplicating it.
3. K applicability/provenance	The qualifications that make the K reduction sound beyond J correspondence: K.BF with established Core.IC persistence, A6, K.OpKey/K.CVPred/K.CV-WF and comparison availability/recursive Core.16 dispositions, K.Drain/Kε/nonwithdrawal, K.EnvMF and JointFreeze where required, CompareAdvanceLegal, Core.CycleClosure/FairPick/FairCycleDovetail/IssueFair/B2/B3/B5, reset-first InterruptionGate/DomResetScope with candidate-specific or uniform U1-U3 discharge, and applicable lowering/waiver/NB-Align conditions. Ordinary/singleton cases may cite the theorem-derived -Ord lemmas where applicable.
4. Policy-S response	For the primary bounded-response A5-L discharge: the SDet-selected Policy-S run and monitor semantics, K.CompleteS, total finite-bound K.RespCov, K.RespClean, response waivers, and CF-S/A5-S, all bound to the same theorem instance. It references rather than duplicates bundle-3 applicability/provenance. With bundle 3, K.2b/K.2c establish A5-L. If another permitted A5-L route is used, its K.A5Cert payload replaces this bundle for that application.

11.3 How the bundles compose by claim

Claim	Bundle composition
Standard QSync functional safety	Bundles 1 + the safety form of 2. J.QSyncTraceInd supplies cycle-aligned packages and Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync supply all-finite-prefix closure for J.IdealPrefixClosed properties. Bundles 3 and 4 are not safety premises. If the K-facing form of bundle 2 already exists, QSyncCertInd-Trace supplies the safety induction.
One selected K.1A cutpoint	The applicable instance qualification, the exact K-facing J/cutpoint package, triggered K applicability evidence, and any valid A5-L discharge. Universal QSync package induction/K.CertCov is not required merely to prove one already selected certified KCut.
Universal Theorem K.1 via Policy-S	All four bundles. Checked QSync structure provides the canonical KCut domain; J.QSyncCertInd -> J.CertExist-QSync -> K.CertCov-QSync provides package totality; bundles 3-4 discharge A5-L through K.2b/K.2c before K.1A-Total.
Universal K result with alternate A5-L route	Bundles 1-3 as triggered plus the alternate K.A5Cert route payload in place of bundle 4; the correspondence, KCut-domain, and K.CertCov obligations remain unchanged by choosing a different proof of A5-L.

This packaging is intentionally an API/user-interface simplification rather than a semantic simplification. A user-facing flow may show four top-level statuses with drill-down to M.7a, but an independent checker must still validate the referenced subordinate evidence and recompute theorem-derived or KObs-derived facts where required.

12. Common misunderstandings

Misreading	Correct interpretation

Misreading	Correct interpretation
“Trace compatible” means cycle accurate.	No. It preserves selected labels, values, ordering, atomicity, channel semantics, and anchors. Independent events may move between cycles.
The source pass transfers by Ref -> Post.	For ordinary safety, the deductive path is Post -> Ref plus J.1-Safe: RTL adds no covered canonical safety behavior absent from Sys_ref. Ref -> Post is restricted to annotated RTL-consistent or witness-selected source prefixes.
Per-process replay certificates already imply a system replay.	No. Shared channel state, snapshots, atomic participants, resets, and field interference can conflict. J.GWR-Comp proves the merge.
The causal-dependency graph is the J induction order.	No. Appendix-G causal dependency explains information flow and may contain legal atomic cycles. J.DepSound applies only to D1-D4 semantic prerequisite edges; D5 is proof stratification and uses J.HorizonOrderSafe commutation evidence. J.FoundationRank/J.DepRank prove the generated J graph acyclic. Appendix K's K.CVPred/K.CV-WF is a third, separately well-founded provenance order and must not reuse the broad causal graph.
KObs is a normalized execution or full state.	No. Core.18 makes the cross-model cutpoint seam equality of the typed record <E,w,H,O>. The source record is defined by Core.18c; Appendix J constructs KObsPost_J and J.KObsConf proves typed equality for the exact selected package. Core.18d/e reconstruct K-facing facts from that record; historical Issue timing and arbitrary RTL/source microstate are not equated.
No-reverse scheduling proves deadlock freedom.	No. It prevents one deadlock-introduction mechanism. Timing overlap, finite capacity, pipeline drain, environment behavior, and serial-shaped source deadlocks require Appendix K.
One clean Policy-S simulation is automatically a proof of A5-S.	No. The primary workflow uses one selected run, but the proof package must also establish SDet, K.Low/K.Low.A5, OpKey/K.CVPred/K.CV-WF including comparison-availability, K.EnvMF with JointFreeze when required, K.CompareAdvanceLegal, Core.CycleClosure/Core.FairPick, reset-first interruption coverage (candidate-specific or uniform U1-U3), total K.RespCov, K.CompleteS, K.RespClean, and exact binding. A universal RTL claim separately needs the QSync/QSyncCertInd package-totality route or another valid K.CertCov discharge.
Core.LAdm adds a new pipeline-drain requirement beyond Appendix B.	No. Core.Phase/Core.LAdm formalize the already-required freeze/drain behavior. Core.16's 'launch' restriction is intentionally broader than message Issue, and K.Drain supplies the finite non-replenishing cleanup evidence; capacity alone does not prove finiteness. Core.SettleKBridge connects the L2 activation to the selected later drain-closed KCut.
The Policy-S reduction is already a certified theorem.	Not as a mechanized and flow-qualified theorem. The scheduling-rules document supplies explicit paper proofs of K.Low.C/K.Low.A5, K.2b.P0/P1/R/P/E/Q and their assembly, including common operation keys, well-founded provenance, typed cycle extension, and nested reset/terminal gates. Lean mechanization, simulator/lowering/QSync qualification, reusable certificate validation, and package-coverage realization remain outstanding.
Another A5-L route can rescue a serial deadlock.	No. If an admissible Policy-S-shaped Policy-L prefix reaches Dead_K^W, A5-L is false for the unchanged instance.

Misreading	Correct interpretation
Snooping changes the specification to match RTL.	No. It selects one behavior already admitted by the under-specified source latency/epsilon semantics.
A live AND is enough for snooping.	No. The wrapper must record per-ordinal RTL events and retain them until source consumption; otherwise events can be dropped or reordered.
Tool generation automatically discharges the proof obligations.	No. The tool or flow must emit documentation, certificates, static checks, monitors, or formal evidence for each named obligation.
A synthesizable and functionally working design is necessarily within the theorem scope.	No. Hidden timing dependence, ambiguous signal anchors, unsupported shared-memory value flow, unmatched reset disposition, fall-through/bypass behavior, or coupled boundary structure can place a correct implementation outside the selected formal model. Appendix F is the architect-facing applicability screen.
A fall-through FIFO is just an ordinary FIFO with a particular capacity.	No. Same-cycle empty-to-consume or full-to-accept behavior changes the channel transition semantics. The bypass or fall-through path must be represented explicitly in Sys_ref and covered by the boundary/elaboration evidence.
Dynamic reset is covered whenever reset is supported by the design.	No. Safety requires matched reset scope/state/request/in-flight disposition. The Policy-S route additionally needs a K.DomResetScope and the prefix-local K.ResetEpoch gate; RW5-disjoint resets can remain legal, but an intersecting reset must either be excluded by Continue or satisfy the independently mapped DirectFail cancellation conditions. Only after Reset Continue is K.TerminalDisposition considered.
Failed non-blocking polls are invisible, so they cannot affect safety.	Their failure is not a committed Sigma event, but the outcome or cadence may select later observable control, values, requests, payloads, or endpoints. Such sites require NB-CF, a valid WC/L.Dir witness, isolation, or an explicitly modeled timing-sensitive protocol.
Input FIFO depths in a joined pipeline are independent available slack.	Not when acceptance is coupled through a join, bundle, shared stage, coupler, or epoch gate. The elaborated boundary model and its occupancy/accounting evidence determine the proof-relevant enablement.
K.Resp is an RTL latency guarantee.	No. It is source-side evidence used to discharge A5-L through serial dominance.
Any epsilon-quiescent state is a KCut.	No. Core.KCut additionally requires reachability and a settled explicit-boundary request/enablement fixed point. Source L2 SettlePoints satisfy this by Core.SettleIsKCut; Core.QSync-Ref supplies their existence/uniqueness in the standard scope, while Core.QSync-Post derives the RTL KCut-closure interface.
K.CertCov covers every RTL prefix or every normalization witness.	No. K.CertCov is total only over reachable RTL KCuts. For each KCut it may exhibit one reachable-prefix/NormPost pair and certified package. In the standard QSync route, QS2a/Core.QSync-KCutCanonical identify the complete canonical KCut domain and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync derive package totality. Arbitrary transient prefixes and arbitrary normalization witnesses need not carry packages.

Misreading	Correct interpretation
QSync all-finite-prefix safety requires the KObs/drain package.	No. J.QSyncTraceInd deliberately carries only exact-prefix J.LocalGWR/J.GWR-Comp plus J.HCanon and excludes OfferReach/OfferConf/KObsConf/K.DrainReach/K.RLAdmReach. It first establishes cycle-aligned packages; Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync then extend J.IdealPrefixClosed safety to all reachable finite outer observable prefixes. If J.QSyncCertInd is already proved for K, it projects to the safety induction.
One observed QSync simulation proves universal K.CertCov.	No. J.QSyncCertInd requires CI1 for every reachable next-cycle successor and CI2 exhaustive successor coverage. A determinism theorem may collapse that set, but B1 alone does not justify ignoring unobserved internal successors.
Delta/tau/idle are just three names for epsilon.	No. Delta is same-cycle settling; a SilentCycle is a real CompleteCycle that may advance registered/environment/monitor state; and a B6 idle tick is permitted only after Core.TerminalIdle proves permanent machine+environment no-future-enabling behavior. A present cycle with no selected commit is not automatically terminal stutter.
Core.QSync makes the source model deterministic.	No. QSync gives a unique settled result for fixed cycle-entry state, inputs, and explicit drives/choices, and QS3 requires the operational settling relation to be realized by that network. RTL QS2a is an origin-recoverability condition for reachable KCuts, not a global execution-determinism theorem. WC-sensitive outcomes, Policy-L latitude, arbitration, and reactive-environment choices may still admit multiple source executions.
Core.KCutClosure_post is still an arbitrary per-instance normalization proof in the standard flow.	No. The downstream interface is unchanged, but the standard QSync scope derives it through Core.QSync-Post from audited structural facts. An alternative non-QSync settling semantics requires an explicitly defined and separately qualified theorem extension.
Same-cycle Core.SyncFence requires a physical RTL sub-cycle ordering.	No. The physical E5 requirement is cycle-granular: the predecessor process-facing commit occurs no later than the synchronization commit. The equal-cycle "accounted before interval advance" rule is the proof/certificate attribution convention used to maintain source-interval semantics.
K.CV can use the Appendix-G causal graph as its induction order.	No. K.CVPred/K.CV-WF defines a separate fact-key provenance relation with explicit predecessor occurrence closure and a well-foundedness/rank certificate. Same-cycle rendezvous enablement and broad causal/temporal proximity do not become provenance edges automatically.
Core.16 'launch' means only endpoint Issue.	No. The 15 August clarification makes launch intentionally broader: any source-later preparation/reach or other work capable of replenishing the blocked component is covered by the no-new-later-work discipline.
“Prefix-closed” means ordinary prefix closure of one linear event log.	No. In the safety theorems it means J.IdealPrefixClosed: downward closure over finite ideals of the retained required-order relation. A cross-interface scoreboard may fail this condition even if an engineer informally calls it a safety check; use direct J.TraceCertCov on the required arbitrary-prefix domain or retain the needed dependency in the observation/order.
Core.IC and Core.SyncFence are trusted axioms that can simply be assumed for RTL.	No. They are constitutive legality/invariant clauses of Sys_ref. The corresponding RTL request lifecycle, attribution, process-facing completion, and synchronization-fence behavior must be established by implementation-

Misreading	Correct interpretation
	side conformance evidence and recorded through the M.7a audit entries.
Commit is defined as valid-and-ready being high in every proof model.	No. Core.8 defines the abstract boundary commit as the legal E4 channel-commit transition. Simultaneous mapped valid/ready and payload sampling are the RTL_B realization/conformance of that abstract event. Commit(q) is the designated process-facing completion/release occurrence, and returned-result availability remains a later Core.Phase L4/Core.RegSeq matter.

13. Quick-reference index

PURPOSE	**Provide a compact map from theorem/definition names to their role in the proof architecture.** **Engineer takeaway:** Use this section as a navigation aid when reading the normative scheduling-rules document.

13.1 Normative Formal Core and Appendix G

Name	Role
Core.8 Commit / CommitCycle	Abstract boundary commit is the selected model's legal E4 channel-commit transition; RTL mapped valid/ready/payload is the implementation realization. Commit(q) is the designated process-facing completion/release occurrence and CommitCycle(q) its cycle index; returned-result availability remains separate under L4/Core.RegSeq.
Core.RegSeq	Registered next-cycle dependence rule: cycle-t sequential interface outputs derive from registered state/fixed operands; no same-cycle newly returned interface result feeds a dependent sequential output.
Core.Settle / delta-settling	Complete source L2 same-cycle settling relation and its fixed-point endpoint. The L2 frame freezes cycle index, registered state, selected cycle inputs, and stable sequential drives; Definition Core.Settle itself does not prove termination.
Core.QSync / QSync-Ref / QSync-Post	Qualified one-global-clock same-cycle model. QS1-QS5 make settling finite, deterministic, acyclic over explicit state boundaries, boundary-complete, and QS3-realized by the selected operational relation. For the standard RTL K-facing scope, QS2a recovers a unique same-cycle K-origin/frame for every reachable post KCut. QSync-Ref supplies source SettlePoint/KCut existence; QSync-Post supplies the canonical RTL endpoint/closure; QSync-KCutCanonical identifies the complete reachable KCut domain.
Core.ReachPrep / Core.IC / Core.PolicyL / Core.LLegal	Overlap-aware source preparation/reach, constitutive ordinary blocking-request persistence/attribution, the permissive issue policy, and exact fixed-instance step legality. Reach may precede an earlier blocking return only when dependency-independent and certified; it remains distinct from Issue. Core.IC is source semantics; RTL conformance is separately checked.

Name	Role
Core.Phase / Core.LAdm / Core.CycleResult / Core.RTLCycleCarrier / Core.CycleClosure / Core.FairPick	L0-L4 phase structure and liveness-admissible source prefixes/executions; typed real-cycle state/result; Core.RTLCycleCarrier completion of an already-entered legal RTL cycle to its successor K-origin; source partial-cycle completion used for complete continuations; separate IssueFair/B2/B3 behavioral fairness; and Core.FairPick/Core.FairCycleDovetail as constructive selector/dovetail evidence when K.2b.E builds the continuation. RTLCycleCarrier is not generic RTL progress.
Core.PolicyS	Conservative serial-blocking issue policy selecting the primary instrumented liveness run.
Core.ElabProj / Core.WFG / Core.16 / Core.Drain / Core.SettleKBridge	Elaboration projection/accounting, authoritative multi-frontier wait-for graph, L2-triggered freeze/no-new-later-work rule, finite non-replenishing drain qualification, and bridge to a later selected drain-closed KCut. Core.16 'launch' includes replenishing preparation/reach as well as message Issue; drain finiteness is a premise, not a consequence of capacity alone.
Core.OrdFrontSingleton	Ordinary non-elaborated source intervals have at most one pending blocking frontier item; genuine multi-frontiers require explicit Sys_B^elab partial-order structure.
Core.18 / Core.18c-e KObs / Appendix C Issue & ReturnedAtCycle	Core.18 defines cross-model correspondence as equality of one common typed KObs record <E,w,H,O>; Core.18c defines the source record and Core.18d/e reconstruction functions. J defines KObsPost_J and proves equality through J.KObsConf. Appendix-C Issue/Commit notation abbreviates relational existence/uniqueness; continued-request attribution may span inserted latency, and Issue/ReturnedAtCycle histories are not KObs fields.
Core.17 ΣMatch / R1-R5 / E1-E5 / causal dependency	Core.17 supplies canonical complete selected occurrence/value matching. Appendix G expands it with R1-R5/E1-E5. R2C fixed-point units match by component identity/local emitted-occurrence ordinal rather than a global cycle bucket; explicitly selected proof-internal RESET units may be carried without adding RESET to Σ. The explanatory causal graph may be cyclic and is neither the J construction order nor K.CV provenance order.
Core.KCut / Core.KCutClosure_post / Core.KCutObserve	K-facing settled/quiescent comparison domain; downstream RTL observation-domain interface; and persistent-blocker preservation into that domain. In the standard scope QSync-Post derives KCutClosure_post; K.CertCov remains separate package totality.
Core.SyncFence	Constitutive Sys_ref synchronization-completion legality with a separately checked RTL conformance counterpart: blocking operations in pref_S(s) process-facing commit no later than s; same-cycle completion is permitted and equal-cycle interval accounting is certificate semantics rather than physical sub-cycle timing.
Core.MemSyncOrder	Array/synchronization commit-order qualification at the schedule/mapping layer; separate from Appendix I/J replay and from J.Mem cross-process shared-memory lowering.

13.2 Appendix I

Name	Role
I.A0	RTL E3 message-issue discipline assumption.

Name	Role
I.ReplayObs	Logical replay checkpoint determined by committed history, stimulus, and witness; excludes raw preparation and current request state.
I.CutpointOfferObs	Operational current-offer slice carrying Pending_P, Front_P, BoundaryReq, attribution, boundary, direction, epoch, and stable Push payload facts.
I.ReplayObs-Congruence	Immediate same-unit replay closure plus paired congruence of the exact selected normalization paths.
I.DR / I.DR'	Deterministic equality of logical replay checkpoints at selected normalized and canonical immediate post-unit cutpoints.
I.DeferredNBHole	Carried unresolved shared non-blocking decision with NoDependentUse.
I.3	Finite process-local preparation script.
I.4	Construction of the J.PCert records.

13.3 Appendix J

Name	Role
J.LocalGWR	Complete local certificate package; no global replay assumption.
J.DepJ / J.DepSound / J.HorizonOrderSafe / DepComplete / DepRank / DepAcyclic	Generate D1-D4 semantic prerequisites and D5 Horizon proof-stratification edges; prove semantic soundness only for D1-D4, validate D5-only pairs by commutation, complete all noncommuting dependencies, and use the graph-independent foundation rank to prove acyclicity.
Finite ideals / J.ValidatedConstructionOrder	Finite canonical ideals/down-closed required-order prefixes used by the J induction; tie ordering among commuting units has no semantic significance. J.RefProjCover uses the same finite-ideal notion for one-run source coverage.
J.RegPhaseNF	Normalize same-Horizon actions to registered L1/L2/L3 phases. It consumes a finite L2-settle existence fact; Core.QSync-Ref supplies it uniformly on the normal compositional route, while a direct J.GWR route may carry per-Horizon evidence.
J.NBDecision	Decide each deferred non-blocking call from the common settled snapshot.
J.UnitExt	Extend one complete Horizon bucket.
J.GWR-Comp	Construct a validated finite-ideal order, reachable J.GWR witness chain, and J.RegPhaseReach from local certificates plus source-side QualifiedSynchronousModel/Core.QSync-Ref on the normal route.
J.1	Top-level finite-prefix observable compatibility over Reach_post/Reach_ref: normal Post -> Ref construction plus a restricted Ref -> Post clause in which J.RTLConsistentRefPrefix already supplies a reachable tau_post* with a complete selected projection matching tau_ref (no unmatched selected unit) and compatible annotations. No generic J.FE/fair-extension premise is used.
J.0	Compose an already selected compatible pair. On normal Post -> Ref, J.GWR-Comp supplies that pair; on restricted Ref -> Post, J.RTLConsistentRefPrefix supplies it.

Name	Role
J.TraceCertCov / J.RefProjCover	Safety-side RTL-prefix certificate coverage and optional one-complete-source-run coverage. J.TraceCertCov is route-sensitive; for CycleReach_post the standard QSync route derives it from J.QSyncTraceInd. Core.RTLCycleCarrier/J.CycleIdealCover can then extend J.IdealPrefixClosed safety to non-cycle-aligned finite ideals without per-prefix packages, while properties that fail ideal-prefix closure still need direct broader-domain J.TraceCertCov. J.RefProjCover uses finite canonical ideals. Neither implies K.CertCov.
J.IdealPrefixClosed / J.1-Safe / J.1-Safe-QSync	J.IdealPrefixClosed defines safety closure under finite required-order ideals, not just suffix deletion from a linear trace. J.1-Safe transfers such a J.HCanon-invariant property through Post -> Ref under route-sensitive J.TraceCertCov. J.1-Safe-QSyncCycle is the cycle-aligned standard-QSync instantiation; J.CycleIdealCover plus J.1-Safe-QSync extend it to every reachable finite RTL prefix. One-run source use additionally requires J.RefProjCover.
J.CycleReach_post / J.QSyncTraceInd / J.CycleIdealCover	Cycle-aligned RTL safety domain and base/one-cycle/exhaustive-successor package induction carrying only LocalGWR/J.GWR-Comp plus J.HCanon; K-only Offer/KObs/drain facts are excluded. J.QSyncCertInd-Trace can supply it. Core.RTLCycleCarrier provides the within-already-entered-cycle execution carrier, and J.CycleIdealCover maps every reachable finite outer canonical history to a finite ideal of a reachable cycle-aligned history.
J.QSyncCertState / J.QSyncCertInd / J.CertExist-QSync	Richer canonical-cutpoint package state and anti-circular base/one-cycle/exhaustive-successor induction. Carries J.HCanon/OfferReach/OfferConf/KObsConf, theorem-derived J.RegPhaseReach, and K.DrainReach. J.CertExist-QSync lifts the induction to total J-side package existence; Appendix K's K.CertCov-QSync binds those tuples into K.CertCov.
Core.18 / J.HCanon / OfferReach / OfferConf / KObsConf / KObsPost_J	Core fixes the common KObs type/equality seam. J proves common canonical history, source H/O realizability, complete RTL request-to-offer attribution, constructs KObsPost_J, and proves its typed equality with KObs_E,w for the exact selected cutpoint package.
Corollary J.1	Export equal KObs plus the J.KObs-Proc/Chan/Offers/WFG reconstruction facts. It makes no K deadlock, waiver, environment-terminal, or diagnostic conclusion.

13.4 Appendix K

Name	Role
K.Dead / K.DeadW	Single authoritative ordinary and waiver-parameterized reachable drain-closed closed internal wait-cycle predicates. A9/A10 refer back to Core endpoint-cardinality/reset-empty conventions rather than restating competing definitions.
K.0 / K.1a	K.0 reconstructs the frontier from KObs-derived inputs. K.1a is the source-side/lowered-KCut disabled-channel characterization used by K.Dead and cites Core.12/Core.13; it is not an arbitrary RTL microstate theorem.
K.KObs-Dead / K.KObs-Ext	K-owned factoring and equal-KObs transfer of deadlock, waiver, environment-terminal, and diagnostic predicates; Core.WFG remains the authoritative graph definition.

Name	Role
K.RLAdmReach	Exact source witness is Core.LAdm-admissible: J.RegPhaseReach plus K.DrainReach, including Core.SettleKBridge, discharging Core.Drain for the same selected KCut package.
K.CertifiedCutpoint	Complete package for one reachable RTL KCut plus one recorded reachable-prefix/NormPost pair ending at that KCut.
K.CertCov	Package totality over every reachable RTL KCut. In the standard QSync route, QS2a/Core.QSync-KCutCanonical identify the canonical domain and J.QSyncCertInd -> J.CertExist-QSync -> K.CertCov-QSync derive total certified packages. Alternate independently checked total-extraction routes remain valid.
SDet	Flow qualification fixing one fully specified deterministic Core.PolicyS simulator/run; separate from run completeness and response cleanliness.
A5-L	No admissible Policy-L prefix reaches a source KCut satisfying Dead_K^W. Drain-closedness is part of Dead_K, not an extra A5-L cutpoint restriction.
K.Low/K.Low.A5 / K.OpKey / K.CVPred/K.CV-WF / K.CompareAdvanceLegal	Lower to literal-blocking Sys_K and transport A5-L back to Sys_ref; provide common cross-policy keys, deterministic/well-founded fact provenance, serial comparison-availability with recursive Core.16 reach dispositions, and the main P0 preparation/R Issue compatibility. Ordinary or comparability-certified singleton cases use K.CompareAdvanceLegal-Ord; explicit multi-frontiers need elaboration/order evidence.
K.JointFreeze-Ord / K.JointFreeze	Frozen-component no-release evidence used by K.EnvMF. Ordinary/non-elaborated or comparability-certified singleton frontiers are theorem-derived by K.JointFreeze-Ord; explicit K.JointFreeze is required only for admitted StandardEnv multi-frontier components.
K.InterruptionGate / K.DomResetScope / K.ResetEpoch / K.TerminalDisposition	Shared reset-first interruption scheme. Either validate candidate-specific K.DomResetScope/DomEpoch + K.ResetEpoch/K.TerminalDisposition Continue/DirectFail branches, or one uniform U1-U3 instance-level record proving all required Continue branches.
K.RespCov / RespClean / CompleteS / Core.CycleClosure / Core.FairPick	Total finite-bound response evidence over one complete selected run, plus typed cycle qualification and the constructive FairPick/FairCycleDovetail evidence used to build the hypothetical fair Policy-L continuation. B2/B3 are the underlying behavioral boundary-progress properties; B5 remains separate.
K.2b / K.2c	Paper-level serial-dominance and A5-S -> A5-L reduction, with K.Low.A5, common keys, K.CV comparison-availability and recursive reach legality, JointFreeze/CompareAdvanceLegal, P0/P1/R/P/E/Q arguments, FairPick/FairCycleDovetail, and candidate-specific or uniform reset-first interruption discharge. Lean mechanization and concrete flow qualification remain pending.
K.1A / K.1A-Total / K.1	K.1A is the per-certified-cutpoint theorem and consumes A5-L but not the serial-route premises. K.1A-Total adds Core.KCutClosure_post and K.CertCov. In the standard QSync route those universal-domain facts are derived through QSync-Post/QSync-KCutCanonical and J.QSyncCertInd/J.CertExist-QSync/K.CertCov-QSync. K.1 is the user-facing composition that first obtains A5-L from SDet/A5-S/K.2b/K.2c and then applies the universal layer.

13.5 Appendix L

Name	Role
L.1	Causal witness selector for epsilon-modeled source choices at generic epsilon-quiescent source choice points. It is intentionally not restricted to Core.KCut, whose additional boundary-evaluation condition is K-specific.
L.2	Conditional coverage/causality/noninterference theorem under which a realizable snoop-selected witness proves WC.
L.3	NB-CF identity witness; no snooping required for irrelevant failed polls.
L.4	Cadence-sensitive polling requires isolation/snoop alignment.
L.Dir	Ordinal-preserving source consumption of a free-running RTL snoop stream; benign cycle skew is allowed, while order/kind/payload divergence means witness non-existence and no theorem-backed guarantee.
L.5	Online harness realization of an existing abstract witness selector, with W1-W5, L.Dir, coverage, and clean failure reporting as separate obligations.
W1-W5	Recorded ordinal observation, source-side commit gating, registered sampling, comparison/watchdog with proved-versus-inconclusive timeout treatment, and separate coverage proof.

13.6 Appendix M evidence packaging

Name	Role
M.7a discharge ledger	Definitive per-instance trigger/discharge/evidence record. Required conditions may not be marked inapplicable; separately named qualification obligations remain explicit even when theorem-derived facts are referenced.
M.7b standard QSync certificate profile	Packaging overlay grouping the M.7a entries into four logical interfaces: instance/QSync qualification, J correspondence/refinement, K applicability/provenance, and Policy-S response. The bundles are not new theorem predicates and do not weaken or merge the underlying obligations.
K.CertHdr / K.CertPkg binding	Common theorem-instance identity and trust boundary tying source/RTL artifacts, capacities/elaboration, observation boundaries, stimulus/environment/reset, witness/waivers, tool/checker versions, and subordinate evidence together. A stronger checked bundle may cite theorem-derived projections rather than generate redundant certificates.

13.7 One-sentence architecture summary

SUMMARY: The proof stack in one sentence

Appendix F provides the architect-facing applicability screen; the Normative Formal Core fixes Sys_ref, Core.8 abstract E4 Commit/Commit(q), the one-global-clock qualified synchronous model, Core.RegSeq/Core.ReachPrep/Core.IC, Policy L/S and Core.IssueFair, Core.Phase/Core.Settle/Core.QSync including RTL QS2a, Core.CycleResult/Core.RTLCycleCarrier/Core.CycleClosure and the constructive Core.FairPick continuation bridge, Core.SyncFence/Core.MemSyncOrder, elaboration/Core.WFG/Core.16/Core.Drain/Core.SettleKBridge, Core.17 ΣMatch, and Core.18 typed KObs equality. Core.IC/Core.SyncFence are source semantics with separate RTL conformance. Appendix G expands R1-R5/E1-E5. Appendix I proves finite process-local replay/preparation. Appendix J builds the finite-prefix source witness and Post -> Ref safety; QSyncTraceInd gives cycle-aligned packages, and Core.RTLCycleCarrier/J.CycleIdealCover/J.1-Safe-QSync extend J.IdealPrefixClosed safety to every reachable finite RTL prefix without per-prefix packages, while the richer QSyncCertInd carries full K-ready packages across cycles. Appendix L conditionally realizes witness-selected epsilon choices. Appendix K factors deadlock from equal KObs, adds Core.LAdm and A5-L, and in the standard QSync route derives the complete canonical KCut domain plus K.CertCov through QSync-KCutCanonical/J.CertExist-QSync/K.CertCov-QSync; K.Dead is authoritative and A9/A10 reference Core conventions. Appendix M.7b may package the M.7a ledger into four logical bundles without changing any premise. The primary one-run Policy-S route uses the K applicability/provenance and response evidence before K.2b/K.2c reduce a clean complete response-monitored run to A5-L. Paper proofs are supplied; Lean mechanization and concrete flow/certificate qualification remain incomplete.

Source note: theorem and definition names follow the scheduling-rules revision through Appendix U dated 18 August 2026.